

Table of Contents

- [Overview](#)
- [Software Requirements](#)
- [Data Understanding](#)
- [Data Preperation](#)
- [Modelling](#)
 - [Pipeline](#)
 - [Experiment - Custom Models](#)
 - [Experiment - Transfer Learning](#)

✓ 1) Overview

Wir, Gruppe 4, haben uns für das Thema 1: "Klassifikation von Bildern aus der industriellen Qualitätskontrolle" entschieden.

Gruppenmitglieder:

- 80751 Simon Ruttman
- 3011613 Marcel Staiger
- 3014537 Tobias Karl

Die Aufgabe haben wir vollständig in diesem Jupyter Notebook behandelt. Für die Entwicklung haben wir Google Colab verwendet. Da das Notebook recht umfangreich ist, empfehlen wir Google Colab, da dieses ein detailliertes Inhaltsverzeichnis im Seitenbereich bereitstellt, mittels dem man gut durch das Notebook navigieren kann.

Das Notebook orientiert sich an den CRISP-DM Phasen, wobei die Evaluationsphase jeweils am Ende eines Lernpfads in der Modell-Phase durchgeführt worden ist. In einer ersten Phase wird der bestehende Datensatz untersucht. Darauf aufbauend wird der Datensatz im Kapitel Data Preperation aufbereitet. Den Kern des Colabs bildet das Kapitel Modelling. In diesem wird zunächst eine kleine Pipeline aufgebaut, damit die Code Duplikation verringert wird. Anschließend sind wir 2 Lernpfade durchliefen. In dem ersten Pfad haben wir uns auf eigene Modelle "from Scratch" konzentriert. Im zweiten Pfad haben wir vortrainierte Modelle verwendet und Transfer-Learning eingesetzt.

Damit das Notebook kleiner ist und schneller lädt, haben wir die Ausgaben entfernt. Eine Version mit allen Ausgaben ist unter diesem [Link](#) verfügbar. Alle Daten und Modelle die wir in diesem Notebook verwenden, sind über Google Drive geshared. Wir greifen auf diese Artefakte mit gdown zu. Die Daten, Modelle, dieses Colab und die Präsentationsfolien befinden sich zudem auf GitHub ([Link](#)).

Dieses Colab ist mit untenstehendem QR-Code oder unter diesem Link erreichbar: [Link](#).



✓ 2) Software Requirements

✓ 2.1 Packages

Dieses Notebook wurde in Google Colab entwickelt. Die Laufzeitumgebung ist **Python 3.12.12**.

Folgende Libraries sind notwendig:

- **numpy**
- **TensorFlow**
- **gdown**
- **Pandas**

```
%%capture
!pip install tensorflow
!pip -q install gdown
!pip install pandas
```

```
!pip freeze
```

► Pip-Log

⚠ **Ggf. ist Laufzeit-Neustart notwendig!**

Menü oben links: **Laufzeit** → **Sitzung neu starten**

✓ 2.2 Hardware Accelerator

⚠ **Einschalten der T4 GPU (optional, notwendig fürs Training)**

Menü oben links **Laufzeit** → **Laufzeit ändern** → wähle **T4 GPU**.

Nur notwendig, wenn man die Modelle trainieren will. Für Visualisierungen etc. reicht die CPU.

```
import tensorflow as tf
print("TF:", tf.__version__)
print("GPUs:", tf.config.list_physical_devices("GPU"))
```

✓ 3) Data Understanding

Wir verwenden den MVTec-Datensatz von [MVTec Software](#). Der Datensatz ist für das Benchmarking von Anomalieerkennungen gedacht. Wir verwenden diesen für die Klassifikation. Der Datensatz wird über diese [Link](#) via Google Drive bereitgestellt.

Das MVTec AD Dataset ist in 15 Objektkategorien unterteilt. Jede Kategorie besitzt einen Trainings- und einen Testordner. Der Trainingsdatensatz enthält ausschließlich fehlerfreie ("good") Bilder, während der Testdatensatz sowohl fehlerfreie als auch defekte Beispiele umfasst. Für jede Defektklasse existieren zusätzlich Pixel-genaue Ground-Truth-Masken im Ordner `ground_truth`, die nur für defekte Bilder vorhanden sind.

MVTec AD Dataset:

```
dataset_root/
├─ bottle/
├─ cable/
├─ capsule/
├─ carpet/
├─ grid/
├─ hazelnut/
├─ leather/
├─ metal_nut/
├─ pill/
├─ screw/
├─ tile/
├─ toothbrush/
├─ transistor/
├─ wood/
└─ zipper/
```

Jede Kategorie hat die gleiche Grundstruktur:

```
<category>/
├─ train/
│   └─ good/           # nur "good" im Training
├─ test/
│   └─ good/           # gute Testbilder
│   └─ <defect_type>/  # Defektklassen (z.B. crack, contamination, ...)
└─ ground_truth/
    └─ <defect_type>/  # Pixel-Masken zu den Defekten (nur für Defekte)
```

✓ 3.1 Dataclass

Um einen Überblick über den verwendeten Datensatz zu erhalten, werden für alle Bilder die relevanten Metadaten erfasst. Diese Metadaten bilden die Grundlage für die weitere Verarbeitung und Analyse.

Folgende Informationen sind relevant:

- Der Dateipfad des Bildes
- Die zugehörige Kategorie (z. B. *cable*)
- Der Status des Bildes (fehlerfrei oder fehlerhaft)
- Der Fehlertyp, abgeleitet aus dem jeweiligen Ordernamen (z. B. *broken_large* in der Kategorie *bottle*)

```
class ImageMetadata:
    def __init__(self, path: str, category: str, is_ok: bool, error_type: str | None = None):
        self.path: str = path
        self.category: str = category
        self.is_ok: bool = bool(is_ok)
        self.error_type: str | None = error_type

    def print(self) -> str:
        status = "IO" if self.is_ok else "NIO"
        err = self.error_type if self.error_type is not None else "none"
        return f"ImageMetadata(path='{self.path}', category='{self.category}', status='{status}', error_type='{err}')
```

✓ 3.1.1 Download dataset

Für dieses Colab wird der MVTec-Datensatz [hier](#) über Google Drive bereitgestellt.

Lädt die Datei `mvtec.tar.xz` aus Google Drive herunter (falls es noch nicht vorhanden ist).

```
import os
import gdown

def download_datasets() -> str:
```

```

data_dir = "/content/data"
os.makedirs(data_dir, exist_ok=True)

file_url = "https://drive.google.com/file/d/1dvMJAcDLnU1GiPZGp-uqwahBbKyGj_0y/view?usp=sharing"
out_path = os.path.join(data_dir, "mvtec_anomaly_detection.tar.xz")

if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
    print(f"Info: File already exists -> skipping ({out_path})")
    return out_path

print("Info: Downloading mvtec_anomaly_detection.tar.xz ...")
gdown.download(
    url=file_url,
    output=out_path,
    quiet=False,
    fuzzy=True,
    use_cookies=True,
)

print(f"Info: Done -> {out_path}")
return out_path

```

✓ 3.1.2 Unzip dataset

Extrahiert das `mvtec_anomaly_detection.tar.xz` Archiv in den `data/` Ordner (falls es noch nicht entpackt wurde).

```

import os
import tarfile

def unzip_datasets() -> None:
    data_dir = "data"
    prefix = "mvtec_anomaly_detection"

    tar_path = os.path.join(data_dir, f"{prefix}.tar.xz")
    out_dir = os.path.join(data_dir, prefix)

    if not os.path.exists(tar_path):
        print(f"Warning: {prefix}: archive not found -> skipping ({tar_path})")
        return

    if os.path.isdir(out_dir) and len(os.listdir(out_dir)) > 0:
        print(f"Info: {prefix}: already extracted -> skipping ({out_dir})")
        return

    os.makedirs(out_dir, exist_ok=True)
    print(f"Info: Extracting {prefix} -> {out_dir}")

    with tarfile.open(tar_path, "r:xz") as t:
        t.extractall(out_dir)

```

3.1.3 Load metainfos to dataclass

Erstellt eine Liste an `ImageMetadata` aus dem extrahierten `mvtec_anomaly_detection.tar.xz` Archiv.

```
from pathlib import Path
from typing import Optional, List

def build_metadata() -> List[ImageMetadata]:
    dataset_root = Path("data") / "mvtec_anomaly_detection"
    root = Path(dataset_root)
    items: List[ImageMetadata] = []

    for category_dir in sorted([p for p in root.iterdir() if p.is_dir()]):
        category = category_dir.name

        train_good_dir = category_dir / "train" / "good"
        test_good_dir = category_dir / "test" / "good"

        if train_good_dir.is_dir():
            for p in sorted(train_good_dir.rglob("*.png")):
                items.append(ImageMetadata(str(p), category, True, None))

        if test_good_dir.is_dir():
            for p in sorted(test_good_dir.rglob("*.png")):
                items.append(ImageMetadata(str(p), category, True, None))

        test_dir = category_dir / "test"
        if test_dir.is_dir():
            for defect_dir in sorted([d for d in test_dir.iterdir() if d.is_dir() and d.name != "good"]):
                defect_type = defect_dir.name
                for p in sorted(defect_dir.rglob("*.png")):
                    items.append(ImageMetadata(str(p), category, False, defect_type))

    return items
```

3.1.4 Execute

```
download_datasets()
unzip_datasets()
data_infos: List[ImageMetadata] = build_metadata()
```

3.2 Analyze Category-ratio and IO/NIO-ratios

Auf Basis der Liste an `ImageMetadata` wird ein Pandas `DataFrame` erstellt, in dem wir analysieren, wie das Verhältnis der Klassen untereinander ist. Pro Klasse untersuchen wir zudem das Verhältnis zwischen IO und NIO.


```

import pandas as pd
from typing import Optional, List

def dataset_summary(data_infos: List[ImageMetaData]) -> pd.DataFrame:
    data_frame = pd.DataFrame([
        "path": info.path,
        "category": info.category,
        "is_ok": info.is_ok,
        "error_type": info.error_type
    ] for info in data_infos])

    amount_total_all: int = len(data_frame)
    summary = []

    # Categories
    for category, group in data_frame.groupby("category"):
        good_dataframe: pd.DataFrame = group[group["is_ok"] == True]
        bad_dataframe: pd.DataFrame = group[group["is_ok"] == False]

        amount_total: int = len(group)
        amount_good: int = len(good_dataframe)
        amount_bad: int = len(bad_dataframe)

        category_share_percent: float = amount_total / amount_total_all * 100.0
        good_share_percent: float = amount_good / amount_total * 100.0
        bad_share_percent: float = amount_bad / amount_total * 100.0

        unique_error_types = ((bad_dataframe["error_type"].dropna()).unique())
        amount_error_types: int = len(unique_error_types)

        summary.append({
            "category": category,
            "category_share_percent": category_share_percent,
            "amount_total": amount_total,
            "amount_good": amount_good,
            "amount_bad": amount_bad,
            "bad_share_percent": bad_share_percent,
            "good_share_percent": good_share_percent,
            "amount_error_types": amount_error_types
        })

    # All categories
    good_dataframe: pd.DataFrame = data_frame[data_frame["is_ok"] == True]
    bad_dataframe: pd.DataFrame = data_frame[data_frame["is_ok"] == False]

    amount_total: int = len(data_frame)
    amount_good: int = len(good_dataframe)
    amount_bad: int = len(bad_dataframe)

    good_share_percent: float = amount_good / amount_total * 100.0
    bad_share_percent: float = amount_bad / amount_total * 100.0

    unique_error_types = ((bad_dataframe["error_type"].dropna()).unique())
    amount_error_types: int = len(unique_error_types)

```

```
summary.append({
    "category": "All",
    "category_share_percent": 100.0,
    "amount_total": amount_total,
    "amount_good": amount_good,
    "amount_bad": amount_bad,
    "bad_share_percent": bad_share_percent,
    "good_share_percent": good_share_percent,
    "amount_error_types": amount_error_types
})

result_frame = pd.DataFrame(summary)
return result_frame
```

```
summary_dataframe = dataset_summary(data_infos)

summary_dataframe_to_display = summary_dataframe.rename(columns={
    "category": "Kategorie",
    "category_share_percent": "Anteil Kategorie (%)",
    "amount_total": "Gesamtanzahl",
    "amount_good": "Anzahl Ok",
    "amount_bad": "Anzahl Fehler",
    "good_share_percent": "Anteil Ok (%)",
    "bad_share_percent": "Anteil Fehler (%)",
    "amount_error_types": "Anzahl Fehlertypen",
})

display(summary_dataframe_to_display.style.format({
    "Anteil Ok (%)": "{:.0f} %",
    "Anteil Fehler (%)": "{:.0f} %",
    "Anteil Kategorie (%)": "{:.0f} %"
}))
```


	Kategorie	Anteil Kategorie (%)	Gesamtanzahl	Anzahl Ok	Anzahl Fehler	Anteil Fehler (%)	Anteil Ok (%)	Anzahl Fehlertypen
0	bottle	5 %	292	229	63	22 %	78 %	3
1	cable	7 %	374	282	92	25 %	75 %	8
2	capsule	7 %	351	242	109	31 %	69 %	5
3	carpet	7 %	397	308	89	22 %	78 %	5
4	grid	6 %	342	285	57	17 %	83 %	5
5	hazelnut	9 %	501	431	70	14 %	86 %	4
6	leather	7 %	369	277	92	25 %	75 %	5
7	metal_nut	6 %	335	242	93	28 %	72 %	4
8	pill	8 %	434	293	141	32 %	68 %	7
9	screw	9 %	480	361	119	25 %	75 %	5
10	tile	6 %	347	263	84	24 %	76 %	5
11	toothbrush	2 %	102	72	30	29 %	71 %	1
12	transistor	6 %	313	273	40	13 %	87 %	4
13	wood	6 %	326	266	60	18 %	82 %	5
14	zipper	7 %	391	272	119	30 %	70 %	7
15	All	100 %	5354	4096	1258	23 %	77 %	48

Insgesamt sind die Klassen im Datensatz weitgehend ausgewogen verteilt. Eine Ausnahme bildet die Kategorie *toothbrush*, da im Vergleich zu anderen Kategorien relativ wenige Beispiele vorhanden sind.

Deutlich kritischer ist die stark schiefe Verteilung zwischen fehlerfreien (*good*) und fehlerhaften (*bad*) Bildern über alle Kategorien hinweg. In nahezu allen Klassen überwiegen die fehlerfreien Beispiele deutlich. Bei der Modellierung muss daher berücksichtigt werden, dass es sich um ein stark unausgeglichenes Klassifikationsproblem handelt.

✓ 3.3 Analyze Error-Category ratios within NIO

Im Folgenden untersuchen wir wie sich die Fehlerkategorien innerhalb einer Kategorie zueinander verhalten. Wir verwenden dafür wieder die Liste an `ImageMetadata` und erstellen uns daraus ein Pandas `DataFrame`.

```
import pandas as pd
from typing import List

def error_type_share_per_category(data_infos: List[ImageMetadata]) -> pd.DataFrame:
    data_frame = pd.DataFrame([
        "path":      info.path,
        "category":  info.category,
        "is_ok":     info.is_ok,
```

```

        "error_type": info.error_type
    } for info in data_infos])

bad_dataframe: pd.DataFrame = data_frame[data_frame["is_ok"] == False].copy()

count_frame: pd.DataFrame = (
    bad_dataframe
    .groupby(["category", "error_type"], dropna=True)
    .size()
    .reset_index(name="amount_error_images")
)

count_frame["amount_error_images_total_category"] = (
    count_frame
    .groupby("category")["amount_error_images"]
    .transform("sum")
)

count_frame["error_type_share_percent_in_category"] = (
    count_frame["amount_error_images"] / count_frame["amount_error_images_total_category"] * 100.0
)

count_frame["amount_error_types_in_category"] = (
    count_frame
    .groupby("category")["error_type"]
    .transform("nunique")
)

count_frame["perfect_balance_share_percent_in_category"] = (
    100.0 / count_frame["amount_error_types_in_category"]
)

count_frame["difference_percent_points"] = (
    count_frame["error_type_share_percent_in_category"]
    - count_frame["perfect_balance_share_percent_in_category"]
)

result_frame: pd.DataFrame = (
    count_frame
    .sort_values(["category", "error_type_share_percent_in_category"], ascending=[True, False])
    .reset_index(drop=True)
)

return result_frame

```

```

error_type_share_dataframe = error_type_share_per_category(data_infos)

error_type_share_dataframe_to_display = error_type_share_dataframe.rename(columns={
    "category": "Kategorie",
    "error_type": "Fehlertyp",
    "amount_error_images": "Anzahl Fehlerbilder",
    "amount_error_images_total_category": "Anzahl Fehlerbilder Gesamt (Kategorie)",
    "error_type_share_percent_in_category": "Anteil Fehlertyp innerhalb Kategorie (%)",
    "amount_error_types_in_category": "Anzahl Fehlertypen (Kategorie)",
})

```

```
"perfect_balance_share_percent_in_category": "Soll-Anteil perfekte Balance (%)",
"difference_percent_points": "Abweichung (Prozentpunkte)",
})

display(error_type_share_dataframe_to_display.style.format({
    "Anteil Fehlertyp innerhalb Kategorie (%)": "{:.1f} %",
    "Soll-Anteil perfekte Balance (%)": "{:.2f} %",
    "Abweichung (Prozentpunkte)": "{:+.2f}",
}))
```

	Kategorie	Fehlertyp	Anzahl Fehlerbilder	Anzahl Fehlerbilder Gesamt (Kategorie)	Anteil Fehlertyp innerhalb Kategorie (%)	Anzahl Fehlertypen (Kategorie)	Soll-Anteil perfekte Balance (%)	Abweichung (Prozentpunkte)
0	bottle	broken_small	22	63	34.9 %	3	33.33 %	+1.59
1	bottle	contamination	21	63	33.3 %	3	33.33 %	-0.00
2	bottle	broken_large	20	63	31.7 %	3	33.33 %	-1.59
3	cable	cut_inner_insulation	14	92	15.2 %	8	12.50 %	+2.72
4	cable	bent_wire	13	92	14.1 %	8	12.50 %	+1.63
5	cable	cable_swap	12	92	13.0 %	8	12.50 %	+0.54
6	cable	missing_cable	12	92	13.0 %	8	12.50 %	+0.54
7	cable	combined	11	92	12.0 %	8	12.50 %	-0.54
8	cable	cut_outer_insulation	10	92	10.9 %	8	12.50 %	-1.63
9	cable	missing_wire	10	92	10.9 %	8	12.50 %	-1.63
10	cable	poke_insulation	10	92	10.9 %	8	12.50 %	-1.63
11	capsule	crack	23	109	21.1 %	5	20.00 %	+1.10
12	capsule	scratch	23	109	21.1 %	5	20.00 %	+1.10
13	capsule	faulty_imprint	22	109	20.2 %	5	20.00 %	+0.18
14	capsule	poke	21	109	19.3 %	5	20.00 %	-0.73
15	capsule	squeeze	20	109	18.3 %	5	20.00 %	-1.65
16	carpet	color	19	89	21.3 %	5	20.00 %	+1.35
17	carpet	thread	19	89	21.3 %	5	20.00 %	+1.35
18	carpet	cut	17	89	19.1 %	5	20.00 %	-0.90
19	carpet	hole	17	89	19.1 %	5	20.00 %	-0.90
20	carpet	metal_contamination	17	89	19.1 %	5	20.00 %	-0.90
21	grid	bent	12	57	21.1 %	5	20.00 %	+1.05
22	grid	broken	12	57	21.1 %	5	20.00 %	+1.05
23	grid	glue	11	57	19.3 %	5	20.00 %	-0.70
24	grid	metal_contamination	11	57	19.3 %	5	20.00 %	-0.70
25	grid	thread	11	57	19.3 %	5	20.00 %	-0.70
26	hazelnut	crack	18	70	25.7 %	4	25.00 %	+0.71
27	hazelnut	hole	18	70	25.7 %	4	25.00 %	+0.71
28	hazelnut	cut	17	70	24.3 %	4	25.00 %	-0.71
29	hazelnut	print	17	70	24.3 %	4	25.00 %	-0.71
30	leather	color	19	92	20.7 %	5	20.00 %	+0.65
31	leather	cut	19	92	20.7 %	5	20.00 %	+0.65
32	leather	glue	19	92	20.7 %	5	20.00 %	+0.65
33	leather	poke	18	92	19.6 %	5	20.00 %	-0.43
34	leather	fold	17	92	18.5 %	5	20.00 %	-1.52
35	metal_nut	bent	25	93	26.9 %	4	25.00 %	+1.88
36	metal_nut	flip	23	93	24.7 %	4	25.00 %	-0.27
37	metal_nut	scratch	23	93	24.7 %	4	25.00 %	-0.27
38	metal_nut	color	22	93	23.7 %	4	25.00 %	-1.34
39	pill	crack	26	141	18.4 %	7	14.29 %	+4.15
40	pill	color	25	141	17.7 %	7	14.29 %	+3.44

41	pill	scratch	24	141	17.0 %	7	14.29 %	+2.74
42	pill	contamination	21	141	14.9 %	7	14.29 %	+0.61
43	pill	faulty_imprint	19	141	13.5 %	7	14.29 %	-0.81
44	pill	combined	17	141	12.1 %	7	14.29 %	-2.23
45	pill	pill_type	9	141	6.4 %	7	14.29 %	-7.90
46	screw	scratch_neck	25	119	21.0 %	5	20.00 %	+1.01
47	screw	manipulated_front	24	119	20.2 %	5	20.00 %	+0.17
48	screw	scratch_head	24	119	20.2 %	5	20.00 %	+0.17
49	screw	thread_side	23	119	19.3 %	5	20.00 %	-0.67
50	screw	thread_top	23	119	19.3 %	5	20.00 %	-0.67
51	tile	glue_strip	18	84	21.4 %	5	20.00 %	+1.43
52	tile	oil	18	84	21.4 %	5	20.00 %	+1.43
53	tile	crack	17	84	20.2 %	5	20.00 %	+0.24
54	tile	gray_stroke	16	84	19.0 %	5	20.00 %	-0.95
55	tile	rough	15	84	17.9 %	5	20.00 %	-2.14
56	toothbrush	defective	30	30	100.0 %	1	100.00 %	+0.00
57	transistor	bent_lead	10	40	25.0 %	4	25.00 %	+0.00
58	transistor	cut_lead	10	40	25.0 %	4	25.00 %	+0.00
59	transistor	damaged_case	10	40	25.0 %	4	25.00 %	+0.00
60	transistor	misplaced	10	40	25.0 %	4	25.00 %	+0.00
61	wood	scratch	21	60	35.0 %	5	20.00 %	+15.00
62	wood	combined	11	60	18.3 %	5	20.00 %	-1.67
63	wood	hole	10	60	16.7 %	5	20.00 %	-3.33
64	wood	liquid	10	60	16.7 %	5	20.00 %	-3.33
65	wood	color	8	60	13.3 %	5	20.00 %	-6.67
66	zipper	broken_teeth	19	119	16.0 %	7	14.29 %	+1.68
67	zipper	split_teeth	18	119	15.1 %	7	14.29 %	+0.84
68	zipper	fabric_border	17	119	14.3 %	7	14.29 %	-0.00
69	zipper	rough	17	119	14.3 %	7	14.29 %	-0.00
70	zipper	combined	16	119	13.4 %	7	14.29 %	-0.84
71	zipper	fabric_interior	16	119	13.4 %	7	14.29 %	-0.84
72	zipper	squeezed_teeth	16	119	13.4 %	7	14.29 %	-0.84

Werden die Fehlerkategorien betrachtet, so zeigt sich, dass diese ziemlich ausbalanziert sind. Eine Ausnahme bildet der Scratch beim Holz. Hier haben wir eine Abweichung von +15%. Das bedeutet, dass diese Klasse innerhalb der Holzfehler etwas überrepräsentiert ist.

3.4 Summary

Für den Datensatz ist vor allem die stark schiefe Verteilung zwischen fehlerfreien und fehlerhaften Bildern über alle Kategorien hinweg kritisch. Die anderen Verteilungen sehen wir im Verhältnis als nicht so kritisch an. Wir fokussieren uns daher in diesem Colab auf die schiefe Verteilung zwischen IO und NIO.

4) Data Preparation

Ziel dieser Phase ist es, den MVTec-Datensatz von [MVTec Software](#) so umzustrukturieren, dass er für die Bildklassifikation nach Kategorie und Zustand (IO, NIO) möglichst einfach eingesetzt werden kann.

In unserem Anwendungsfall soll später der Keras-Ordnerloader

keras.utils.image_dataset_from_directory

 verwendet werden.

Aus diesem Grund wird eine neue Verzeichnisstruktur aufgebaut, in der jede Kategorie in einen **IO**- und einen **NIO**-Ordner aufgeteilt wird.

- Im **IO-Ordner** befinden sich alle fehlerfreien Bilder (*good*) der jeweiligen Kategorie.
- Alle fehlerhaften Bilder einer Kategorie werden gesammelt im **NIO-Ordner** abgelegt.
- Die vorhandenen `ground_truth`-Masken werden in diesem Setup nicht verwendet.
- Sämtliche Bilder werden über alle Kategorien hinweg von ursprünglich ca. **1–1.5 MB** auf etwa **5–10 KB** verkleinert, um die Trainingszeiten auf einem moderaten Niveau zu halten.
- Zusätzlich wird **Data Augmentation** eingesetzt, um das Verhältnis zwischen **NIO**- und **IO**-Beispielen auszugleichen.
- Die Bilder werden in **Training**, **Validierung** und **Test** gesplittet. Wir stratifizieren dabei nach der Fehlerkategorie. Augmentate und deren Originale sind zudem immer im gleichen Split enthalten.

Links:

Original-Ordner [hier](#).

Klassifikations-Ordner komprimiert mit Data Augmentation [hier](#).

Klassifikations-Ordner komprimiert ohne Data Augmentation [hier](#).

Klassifikations-Ordner komprimiert mit Data Augmentation nur nach Kategorie [hier](#).

Klassifikations-Ordner komprimiert ohne Data Augmentation nur nach Kategorie [hier](#).

Ordnerstruktur:

```
dataset_root/
├─ bottle_IO/      │─ bottle_NIO/
├─ cable_IO/       │─ cable_NIO/
├─ capsule_IO/     │─ capsule_NIO/
├─ carpet_IO/      │─ carpet_NIO/
├─ grid_IO/        │─ grid_NIO/
├─ hazelnut_IO/    │─ hazelnut_NIO/
├─ leather_IO/     │─ leather_NIO/
├─ metal_nut_IO/   │─ metal_nut_NIO/
├─ pill_IO/        │─ pill_NIO/
├─ screw_IO/       │─ screw_NIO/
├─ tile_IO/        │─ tile_NIO/
├─ toothbrush_IO/  │─ toothbrush_NIO/
├─ transistor_IO/  │─ transistor_NIO/
├─ wood_IO/        │─ wood_NIO/
└─ zipper_IO/      │─ zipper_NIO/
```

NIO Ordner:

```
- bottle_NIO_broken_large_000.jpg
- bottle_NIO_broken_large_000__aug_01.jpg
- bottle_NIO_broken_large_000__aug_02.jpg
```

```
- bottle_NIO_broken_large_000__aug_03.jpg

- bottle_NIO_broken_large_001.jpg
- bottle_NIO_broken_large_001__aug_01.jpg
- bottle_NIO_broken_large_001__aug_02.jpg
- bottle_NIO_broken_large_001__aug_03.jpg
```

IO Ordner:

```
- bottle_IO_good_000.jpg
- bottle_IO_good_001.jpg
- bottle_IO_good_002.jpg
- bottle_IO_good_003.jpg
```

✓ 4.1 Data Augmentation

Um das Verhältnis zwischen **NIO**- und **IO**-Beispielen auszugleichen, wird im folgenden **Data Augmentation** eingesetzt. Es werden dabei die **NIO** so oft augmentiert, sodass das Verhältnis zwischen **NIO** und **IO** exakt **50:50** ist.

✓ 4.1.1. Augmentation Policy

Nicht jede Augmentation ist für jede Kategorie sinnvoll. Eine falsche Transformationen kann die Bildinhalte verfälschen und künstliche Artefakte erzeugen (z. B. wenn eine Flasche durch Rotation/Interpolation plötzlich „Ecken“ bekommt). Daher wird die Augmentation pro Kategorie bewusst eingeschränkt und nur mit solchen Operationen umgesetzt, die zu einem realistischen Bild führen.

```
from pathlib import Path

DEFAULT_POLICY = {
    # Geometrie
    "rotation_mode": "none",          # 'none' | 'small'
    "rotation_limit_deg": 0,
    "flip_strategy": None,            # None | 'h' | 'v' | 'one_of_hv'

    # Farbe / Belichtung
    "use_brightness_contrast": False,
    "brightness_limit": 0.0,
    "contrast_limit": 0.0,
    "brightness_p": 0.0,

    "use_hsv": False,
    "hue_shift_limit": 0,
    "sat_shift_limit": 0,
    "val_shift_limit": 0,
    "hsv_p": 0.0,

    "use_gamma": False,
```



```

"gamma_limit": (100, 100),
"gamma_p": 0.0,
}

POLICY = {
# ----- kleine Rotation -----
"bottle": {"rotation_mode": "small", "rotation_limit_deg": 10},
"cable": {"rotation_mode": "small", "rotation_limit_deg": 10},

# ----- leichter Farb-/Belichtungs jitter -----
"capsule": {
    "use_brightness_contrast": True,
    "brightness_limit": 0.08,
    "contrast_limit": 0.08,
    "brightness_p": 1.0,
    "use_hsv": True,
    "hue_shift_limit": 5,
    "sat_shift_limit": 8,
    "val_shift_limit": 8,
    "hsv_p": 1.0
},

"pill": {
    "use_brightness_contrast": True,
    "brightness_limit": 0.08,
    "contrast_limit": 0.08,
    "brightness_p": 1.0,
    "use_hsv": True,
    "hue_shift_limit": 5,
    "sat_shift_limit": 10,
    "val_shift_limit": 8,
    "hsv_p": 1.0
},

# ----- Rotation + Farbe -----
"screw": {
    "rotation_mode": "small",
    "rotation_limit_deg": 10,
    "use_brightness_contrast": True,
    "brightness_limit": 0.05,
    "contrast_limit": 0.05,
    "brightness_p": 1.0
},

# ----- Spiegelung zufällige Achse -----
"carpet": {"flip_strategy": "one_of_hv"},
"grid": {"flip_strategy": "one_of_hv"},
"hazelnut": {"flip_strategy": "one_of_hv"},
"metal_nut": {"flip_strategy": "one_of_hv"},

# ----- Spiegelung Y-Achse -----
"toothbrush": {"flip_strategy": "h"},
"transistor": {"flip_strategy": "h"},
"wood": {"flip_strategy": "h"},
"zipper": {"flip_strategy": "h"},

```

```

# Default
"leather": {},
"tile": {}
}

def policy_for(category: str) -> dict:
    # Defaults + spezifische Overrides (Overrides gewinnen)
    return {**DEFAULT_POLICY, **POLICY.get(category, {})}

```

✓ 4.1.2 Build Augmentationpipeline from policy

Baut eine Augmentierungspipeline unter der Verwendung der Policy für die Kategorie

```

import albumentations as A
import cv2

def build_transform_for_category_from_policy(category: str) -> A.Compose:
    policy = policy_for(category)
    augmentation_steps = []

    # --- Rotation ---
    rotation_mode = policy.get("rotation_mode", "none")

    if rotation_mode == "discrete_90":
        rotation_transform = A.RandomRotate90(p=1.0)
        augmentation_steps.append(rotation_transform)

    elif rotation_mode == "small":
        rotation_limit_degrees = policy.get("rotation_limit_deg", 0)
        rotation_transform = A.Rotate(
            limit=rotation_limit_degrees,
            border_mode=cv2.BORDER_REFLECT_101,
            p=1.0
        )
        augmentation_steps.append(rotation_transform)

    elif rotation_mode == "medium":
        rotation_limit_degrees = max(int(policy.get("rotation_limit_deg", 20)), 1)
        rotation_transform = A.Rotate(
            limit=rotation_limit_degrees,
            border_mode=cv2.BORDER_REFLECT_101,
            p=1.0
        )
        augmentation_steps.append(rotation_transform)

    elif rotation_mode == "free":
        rotation_transform = A.Rotate(limit=180, border_mode=cv2.BORDER_REFLECT_101, p=1.0)
        augmentation_steps.append(rotation_transform)

```



```
# --- Flips ---
flip_strategy = policy.get("flip_strategy")
flip_probability = policy.get("flip_prob", 1.0)

if flip_strategy == "one_of_hv":
    horizontal_flip = A.HorizontalFlip(p=1.0)
    vertical_flip = A.VerticalFlip(p=1.0)
    flip_transform = A.OneOf([horizontal_flip, vertical_flip], p=1.0)
    augmentation_steps.append(flip_transform)

elif flip_strategy == "h":
    flip_transform = A.HorizontalFlip(p=flip_probability)
    augmentation_steps.append(flip_transform)

elif flip_strategy == "v":
    flip_transform = A.VerticalFlip(p=flip_probability)
    augmentation_steps.append(flip_transform)

# --- Farb-/Belichtungsvariationen ---
use_brightness_contrast = policy.get("use_brightness_contrast", False)
brightness_probability = policy.get("brightness_p", 0.0)

if use_brightness_contrast and brightness_probability > 0:
    brightness_transform = A.RandomBrightnessContrast(
        brightness_limit=policy.get("brightness_limit", 0.0),
        contrast_limit=policy.get("contrast_limit", 0.0),
        p=brightness_probability
    )
    augmentation_steps.append(brightness_transform)

use_hsv = policy.get("use_hsv", False)
hsv_probability = policy.get("hsv_p", 0.0)

if use_hsv and hsv_probability > 0:
    hsv_transform = A.HueSaturationValue(
        hue_shift_limit=policy.get("hue_shift_limit", 0),
        sat_shift_limit=policy.get("sat_shift_limit", 0),
        val_shift_limit=policy.get("val_shift_limit", 0),
        p=hsv_probability
    )
    augmentation_steps.append(hsv_transform)

use_gamma = policy.get("use_gamma", False)
gamma_probability = policy.get("gamma_p", 0.0)

if use_gamma and gamma_probability > 0:
```

```

gamma_transform = A.RandomGamma(
    gamma_limit=policy.get("gamma_limit", (100, 100)),
    p=gamma_probability
)
augmentation_steps.append(gamma_transform)

# Keine Blur/Noise/Perspektive - bewusst ausgelassen, um Defekte zu erhalten
augmentation_pipeline = A.Compose(augmentation_steps)
return augmentation_pipeline

```

✓ 4.1.3 Generate augmentation from original

Nimmt ein ImageMetaData, liest das Bild, augmentiert es anhand der Kategorie-Policy und speichert es in die gleiche relative Ordnerstruktur ins Augmentation-Verzeichnis `mvtec_anomaly_detection_augmentation`.

Beispiel:

```
data/mvtec_anomaly_detection/bottle/test/broken_large/000.png
```

wird augmentiert und hier abgelegt

```
data/mvtec_anomaly_detection_augmentation/bottle/test/broken_large/000__aug.png
```

```

from pathlib import Path
import cv2

def augment_single_image(image_info: ImageMetaData) -> Path:

    # Determine aug path
    source_root: Path = Path("data") / "mvtec_anomaly_detection"
    target_root: Path = Path("data") / "mvtec_anomaly_detection_augmentation"

    input_image_path = Path(image_info.path)
    relative_path = input_image_path.relative_to(source_root)

    output_image_path = target_root / relative_path
    output_image_path = next(p for p in (output_image_path.with_name(f"{output_image_path.stem}__aug_{i:02d}{output_image_path.suffix}") for i in range(1, 10)) if not p.exists())

    output_image_path.parent.mkdir(parents=True, exist_ok=True)

    # Pipeline of category
    augmentation_pipeline = build_transform_for_category_from_policy(image_info.category)

    # Read
    input_image_bgr = cv2.imread(str(input_image_path), cv2.IMREAD_COLOR)
    input_image_rgb = cv2.cvtColor(input_image_bgr, cv2.COLOR_BGR2RGB)

    # Augment
    augmentation_result = augmentation_pipeline(image=input_image_rgb)

```

```

augmented_image_rgb = augmentation_result["image"]
augmented_image_bgr = cv2.cvtColor(augmented_image_rgb, cv2.COLOR_RGB2BGR)

# Write
write_successful = cv2.imwrite(str(output_image_path), augmented_image_bgr)
if not write_successful:
    raise IOError(f"Could not write image: {output_image_path}")

return output_image_path

```

✓ 4.1.4 Augment to 50:50

Für jede Kategorie werden so viele augmentierte NIO-Bilder erzeugt, bis NIO == IO (50:50). Dafür wird über die Liste der fehlerhaften Bilder zyklisch iteriert und jedes Bild bei Bedarf mehrfach augmentiert.

```

def augment_to_50_50(items: List[ImageMetadata]) -> None:
    categories: List[str] = []
    for item in items:
        if item.category not in categories:
            categories.append(item.category)

    for category in categories:
        # Select items of category
        items_of_category: List[ImageMetadata] = []
        for item in items:
            if item.category == category:
                items_of_category.append(item)

        # Determine good count and bad count
        good_count = 0
        bad_count = 0
        for item in items_of_category:
            if item.is_ok is True:
                good_count += 1
            else:
                bad_count += 1

        bad_items_of_category: List[ImageMetadata] = []
        for item in items_of_category:
            if item.is_ok is False:
                bad_items_of_category.append(item)

        while bad_count < good_count:
            for item in bad_items_of_category:
                if bad_count == good_count:
                    break

                augment_single_image(item)
                bad_count += 1

```

```
from collections import Counter

counts = Counter(item.is_ok for item in data_infos if item.category == "bottle")
print("bottle IO (True):", counts.get(True, 0))
print("bottle NIO (False):", counts.get(False, 0))
```

```
augment_to_50_50(data_infos)
```

✓ 4.1.5 Combine Folders

```
from pathlib import Path
import shutil

def merge_mvtec_and_augmentation() -> Path:
    source_root = Path("data") / "mvtec_anomaly_detection"
    augmentation_root = Path("data") / "mvtec_anomaly_detection_augmentation"
    merged_root = Path("data") / "mvtec_anomaly_detection_merged"

    shutil.copytree(source_root, merged_root, dirs_exist_ok=True)
    shutil.copytree(augmentation_root, merged_root, dirs_exist_ok=True)

    return merged_root
```

```
merge_mvtec_and_augmentation()
```

✓ 4.2 Apply new folder structure

Liest Bilder und kopiert diese in die neue Ordnerstruktur.

Namensschema:

```
<category>_<IO|NIO>_<error_type>_<original_filename>
```

Beispiel:

```
bottle/test/broken_small/000.png
-> dataset_root/bottle_NIO/bottle_NIO_broken_small_000.png

bottle/test/broken_small/000__aug_01.png
-> dataset_root/bottle_NIO/bottle_NIO_broken_small_000__aug_01.png
```

```
from pathlib import Path
import shutil

def apply_classification_structure(
    merged_root: Path,
    dataset_root: Path,
```

```

        file_extension: str = ".png",
    ) -> None:
        dataset_root.mkdir(parents=True, exist_ok=True)

        for image_path in merged_root.rglob(f"*{file_extension}"):
            if "ground_truth" in image_path.parts:
                continue

            relative_path = image_path.relative_to(merged_root)
            if len(relative_path.parts) < 4:
                continue

            category = relative_path.parts[0]
            split = relative_path.parts[1]
            error_type = relative_path.parts[2]

            status = "IO" if error_type == "good" else "NIO"
            target_folder = dataset_root / f"{category}_{status}"

            target_name = ""
            if error_type == "good":
                # ...avoid overwriting from test/good to train/good...
                target_name = f"{category}_{status}_{split}_{error_type}_{image_path.name}"
            else:
                target_name = f"{category}_{status}_{error_type}_{image_path.name}"

            target_folder.mkdir(parents=True, exist_ok=True)
            target_path = target_folder / target_name

            shutil.copy2(image_path, target_path)

```

```

merged_root: Path = Path("data") / "mvtec_anomaly_detection_merged"
dataset_root: Path = Path("data") / "mvtec_anomaly_detection_augmentation_classification"

apply_classification_structure(merged_root, dataset_root)

```

```

merged_root: Path = Path("data") / "mvtec_anomaly_detection"
dataset_root: Path = Path("data") / "mvtec_anomaly_detection_classification"

apply_classification_structure(merged_root, dataset_root)

```

✓ 4.3 Reduce Datasize

Wir reduzieren Bilder von ursprünglich ca. **1–1.5 MB** auf etwa **5–10 KB**.

```

from pathlib import Path
from PIL import Image

def compress_images(in_root,out_root):
    in_root = Path(in_root)

```

```

out_root = Path(out_root)
out_root.mkdir(parents=True, exist_ok=True)

for src in in_root.rglob("*"):
    if not src.is_file():
        continue
    if src.suffix.lower() not in [".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff"]:
        continue

    rel = src.relative_to(in_root)
    out_path = (out_root / rel).with_suffix(".jpg")
    out_path.parent.mkdir(parents=True, exist_ok=True)

    with Image.open(src) as im:
        im = im.convert("RGB").resize((180, 180))
        im.save(out_path, "JPEG", quality=70)

```

```

compress_images("data/mvtec_anomaly_detection_augmentation_classification",
               "data/mvtec_anomaly_detection_augmentation_classification_compressed");

```

```

compress_images("data/mvtec_anomaly_detection_classification",
               "data/mvtec_anomaly_detection_classification_compressed");

```

✓ 4.4 Split into Training, Validation and Test

Datensplit

Wir haben uns für drei Datensplits entschieden: **Training**, **Validierung** und **Test**. Der Trainingssplit wird zum Lernen verwendet. Der Validierungssplit wird während des Trainings zur Überwachung durch Callbacks verwendet und auch zwischen den Trainings von uns als Entwicklern als Vergleichsgröße verwendet. Dadurch optimieren wir indirekt auf den Validierungsdatensatz. Wir verwenden daher für die Evaluation einen separaten Testdatensatz.

Vorsicht ⚠ Da wir augmentierte Daten verwenden, haben wir darauf geachtet, dass in der Splitlogik eine Gruppe von (Originalbild + Augmentate) entweder im Trainings-, im Validierungs- oder im Testdatensatz vorkommt. D.h. es ist unterbunden, dass man im Training auf ein Originalbild trainiert und man gegen ein augmentat dieses Bildes später in der Evaluierungsphase testet. Das Verhältnis zwischen IO und NIO ist in allen Splits nahezu 50:50.

Vorsicht ⚠ Es gibt verschiedene Fehlergruppen. Um einen sauberen Split zu erreichen, achten wir darauf, dass wir nach der Fehlerkategorie stratifiziert splitten.

```

import random, shutil
from pathlib import Path
from dataclasses import dataclass, field
from typing import List

# 1. Build

```



```

# Error => Aug Group => Files of augmentation
# E.g.    broken_large
#        => broken_large_000,                                broken_large_001
#        => broken_large_000.jpg, broken_large_000__aug_01.jpg, broken_large_001.jpg, broken_large_001__aug_01.jpg
# 2. Split for each error by aug group
@dataclass
class AugGroup:
    name: str
    files: List[Path] = field(default_factory=list)

@dataclass
class ErrorGroup:
    name: str
    aug_groups: List[AugGroup] = field(default_factory=list)

def nio_error_type(aug_group_name: str, class_name: str) -> str:
    rest = aug_group_name[len(class_name) + 1:]
    parts = rest.split("_")
    return "_".join(parts[:-1])

def split_grouped_dataset_stratified(
    source_folder: Path,
    output_folder: Path,
    train_ratio=0.8,
    val_ratio=0.1,
    test_ratio=0.1,
    seed=42
):
    rng = random.Random(seed)

    for class_folder in source_folder.iterdir():
        error_groups: List[ErrorGroup] = []

        # Build hierarchy: ErrorGroup -> AugGroup -> Files
        for file_path in class_folder.iterdir():
            aug_group_name = file_path.stem.split("__aug_", 1)[0]

            error_name = "ALL"
            if class_folder.name.endswith("_NIO"):
                error_name = nio_error_type(aug_group_name, class_folder.name)

            # Find error or create
            current_error_group = None
            for eg in error_groups:
                if eg.name == error_name:
                    current_error_group = eg
                    break
            if current_error_group is None:
                current_error_group = ErrorGroup(error_name)
                error_groups.append(current_error_group)

            # Find aug group or create
            current_aug_group = None
            for ag in current_error_group.aug_groups:
                if ag.name == aug_group_name:

```

```

        current_aug_group = ag
        break
    if current_aug_group is None:
        current_aug_group = AugGroup(aug_group_name)
        current_error_group.aug_groups.append(current_aug_group)

    # Add to aug group
    current_aug_group.files.append(file_path)

# Split per error group (stratified)
train_aug_groups, val_aug_groups = set(), set()

for error_group in error_groups:
    aug_group_names = [ag.name for ag in error_group.aug_groups]
    rng.shuffle(aug_group_names)

    n = len(aug_group_names)
    n_train = int(round(train_ratio * n))
    n_val = int(round(val_ratio * n))

    train_aug_groups.update(aug_group_names[:n_train])
    val_aug_groups.update(aug_group_names[n_train:n_train + n_val])

# Copy files
for error_group in error_groups:
    for aug_group in error_group.aug_groups:
        split = (
            "train" if aug_group.name in train_aug_groups
            else "val" if aug_group.name in val_aug_groups
            else "test"
        )
        for file_path in aug_group.files:
            dst = output_folder / split / class_folder.name / file_path.name
            dst.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(file_path, dst)

```

```

split_grouped_dataset_stratified(Path(r"data/mvtec_anomaly_detection_augmentation_classification_compressed"),
                                Path(r"data/mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test"), 0.8, 0.1, 0.1)

```

```

split_grouped_dataset_stratified(Path(r"data/mvtec_anomaly_detection_classification_compressed"),
                                Path(r"data/mvtec_anomaly_detection_classification_compressed_train_val_test"), 0.8, 0.1, 0.1)

```

4.5 Merge for category classification only

```

from pathlib import Path
import shutil

def merge_io_nio_to_new_folder(source_root: Path, output_root: Path):
    if output_root.exists():
        shutil.rmtree(output_root)

```



```

shutil.copytree(source_root, output_root)

for split_name in ("train", "val", "test"):
    split_folder = output_root / split_name

    for class_folder in split_folder.iterdir():
        name = class_folder.name
        base = name[:-3] if name.endswith("_IO") else name[:-4] if name.endswith("_NIO") else None
        if base is None:
            continue

        target_folder = split_folder / base
        target_folder.mkdir(exist_ok=True)

        for image_file in class_folder.iterdir():
            shutil.copy2(image_file, target_folder / image_file.name)

    for class_folder in split_folder.iterdir():
        if class_folder.name.endswith("_IO") or class_folder.name.endswith("_NIO"):
            shutil.rmtree(class_folder)

```

```

merge_io_nio_to_new_folder(Path(r"data/mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test"),
                           Path(r"data/mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test_categories_only"))

```

```

merge_io_nio_to_new_folder(Path(r"data/mvtec_anomaly_detection_classification_compressed_train_val_test"),
                           Path(r"data/mvtec_anomaly_detection_classification_compressed_train_val_test_categories_only"))

```

✓ 4.6 Export

Komprimierte Ordner klassifiziert nach Kategorie und IO/NIO. Einmal mit augmentierten Bildern und einmal ohne augmentierten Bildern downloaden.

```

from pathlib import Path
from google.colab import files
import shutil

folder = Path("data") / "mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test"
zip_base = "mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test"

shutil.make_archive(zip_base, "zip", root_dir=str(folder))
files.download(zip_base + ".zip")

```

```

from pathlib import Path
from google.colab import files
import shutil

folder = Path("data") / "mvtec_anomaly_detection_classification_compressed_train_val_test"
zip_base = "mvtec_anomaly_detection_classification_compressed_train_val_test"

```

```
shutil.make_archive(zip_base, "zip", root_dir=str(folder))
files.download(zip_base + ".zip")
```

Komprimierte Ordner klassifiziert nur nach Kategorie. Einmal mit augmentierten Bildern und einmal ohne augmentierten Bildern downloaden.

```
from pathlib import Path
from google.colab import files
import shutil

folder = Path("data") / "mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test_categories_only"
zip_base = "mvtec_anomaly_detection_augmentation_classification_compressed_train_val_test_categories_only"

shutil.make_archive(zip_base, "zip", root_dir=str(folder))
files.download(zip_base + ".zip")
```

```
from pathlib import Path
from google.colab import files
import shutil

folder = Path("data") / "mvtec_anomaly_detection_classification_compressed_train_val_test_categories_only"
zip_base = "mvtec_anomaly_detection_classification_compressed_train_val_test_categories_only"

shutil.make_archive(zip_base, "zip", root_dir=str(folder))
files.download(zip_base + ".zip")
```

✓ 4.7 Download

Die eben heruntergeladenen Ordner werden über Google Drive zur Verfügung gestellt und können mit dieser Methode in die Google Colab Runtime geladen werden. Dadurch müssen wir nach einem Neustart der Colab Runtime nicht wieder die ersten Phasen durchlaufen, sondern können direkt die aufbereiteten Daten downloaden und mit der Modellierung starten.

```
import os
import zipfile
import gdown

def load(file_url: str, prefix: str) -> str:
    data_dir: str = "/content/data"
    os.makedirs(data_dir, exist_ok=True)

    zip_path = os.path.join(data_dir, f"{prefix}.zip")
    out_dir = os.path.join(data_dir, prefix)

    # Download
    if os.path.exists(zip_path) and os.path.getsize(zip_path) > 0:
        print(f"Info: File already exists -> skipping download ({zip_path})")
    else:
        print("Info: Downloading archive ...")
        gdown.download(
```

```

        url=file_url,
        output=zip_path,
        quiet=False,
        fuzzy=True,
        use_cookies=True,
    )
    if not (os.path.exists(zip_path) and os.path.getsize(zip_path) > 0):
        raise RuntimeError(f"Download failed or produced empty file: {zip_path}")
    print(f"Info: Done -> {zip_path}")

# Extract
if os.path.isdir(out_dir) and len(os.listdir(out_dir)) > 0:
    print(f"Info: Already extracted -> skipping ({out_dir})")
    return out_dir

os.makedirs(out_dir, exist_ok=True)
print(f"Info: Extracting {prefix} -> {out_dir}")

with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall(out_dir)

print("Info: Extraction done.")
return out_dir

```

```

file_url: str = "https://drive.google.com/file/d/1WwMxiOZ5RM-WUKqgj0C0cNpepw8u08c/view?usp=sharing"
prefix: str = "data_augmented_split"
load(file_url, prefix)

```

```

file_url: str = "https://drive.google.com/file/d/11jGwy7wVKnn00eQ6a1Wc1-bgOMm7PAMX/view?usp=sharing"
prefix: str = "data_not_augmented_split"
load(file_url, prefix)

```

```

file_url: str = "https://drive.google.com/file/d/18XJAFWiyFQJtasa39Xfv10e8e-xMgkX0/view?usp=sharing"
prefix: str = "data_augmented_split_categories_only"
load(file_url, prefix)

```

```

file_url: str = "https://drive.google.com/file/d/1RwlzUVk6AiEHyHj7q8XXNygVh9E7tcPt/view?usp=sharing"
prefix: str = "data_not_augmented_split_categories_only"
load(file_url, prefix)

```

5) Modelling

Die Modellierung splittet sich in zwei Lernpfade. Dem entwickeln und trainieren von eigenen Modellen, sowie dem verwenden von vortrainierten Modellen.

Wir gehen davon aus, dass vortrainierte Modelle mit Transferlearning bessere Ergebnisse liefern werden, dennoch haben wir uns - gerade zu Lernzwecken - dazu entschieden zunächst Custom Modelle zu erstellen und von Grund auf zu trainieren. Im Anschluss daran haben wir uns im zweiten Pfad mit den vortrainierten Modellen auseinander gesetzt.

✓ 5.1 Pipeline

Ziel dieser Phase ist es, eine kleine Pipeline aufzubauen, mit der verschiedene Modelle einfach trainiert, evaluiert, exportiert und importiert werden können.

✓ 5.1.1 Training

Wir verwenden 3 Callbacks während es Trainings

- Early Stopping

Beendet das Training, wenn sich der Validierungsfehler über mehrere Epochen nicht verbessert. Dies reduziert die Trainingszeit erheblich und schützt uns zudem vor Overfitting.

- Model Checkpoint

Speichert das Modell mit dem kleinsten Validierungsfehler.

- Learning Rate Reducer

Reduziert die Lernrate, wenn eine Verbesserung ausbleibt.

```
from dataclasses import dataclass
from typing import Dict, Optional, List
import numpy as np
import tensorflow as tf
from tensorflow import keras

@dataclass
class TrainInput:
    data_dir: str
    model: tf.keras.Model
    batch_size: int = 32
    seed: int = 42
    epochs: int = 50
    class_weight: Optional[Dict[int, float]] = None
    tensorboard_logdir: Optional[str] = None
    callback_metric: str = "val_loss"
    class_names: Optional[List[str]] = None

@dataclass
class TrainOutput:
    model: tf.keras.Model
    hist: keras.callbacks.History

def train(inp: TrainInput) -> TrainOutput:
    tf.keras.utils.set_random_seed(inp.seed)
```

```
training_train_tensor = tf.keras.utils.image_dataset_from_directory(
    inp.data_dir+"/train",
    seed=inp.seed,
    image_size=(180, 180),
    batch_size=inp.batch_size,
    class_names=inp.class_names
)

training_eval_tensor = tf.keras.utils.image_dataset_from_directory(
    inp.data_dir+"/val",
    seed=inp.seed,
    image_size=(180, 180),
    batch_size=inp.batch_size,
    class_names=inp.class_names
)

callbacks = [
    # Stops earlier than specified epochs, when no longer a
    # significant improve on 'val_loss' occurs for 10 epochs
    keras.callbacks.EarlyStopping(
        monitor=inp.callback_metric,
        patience=10,
        restore_best_weights=True,
        verbose=1
    ),
    # Saves the best model based on val_loss
    keras.callbacks.ModelCheckpoint(
        f"{inp.model.name}.keras",
        monitor=inp.callback_metric,
        save_best_only=True
    ),
    # Reduces learning rate when a val_loss has stopped improving.
    keras.callbacks.ReduceLROnPlateau(
        monitor=inp.callback_metric,
        factor=0.1,
        patience=5,
        verbose=1
    )
]

if inp.tensorboard_logdir:
    callbacks.append(
        keras.callbacks.TensorBoard(
            log_dir=inp.tensorboard_logdir,
            histogram_freq=0,
            write_graph=True
        )
    )

hist = inp.model.fit(
    training_train_tensor,
    validation_data=training_eval_tensor,
    epochs=inp.epochs,
    callbacks=callbacks,
    verbose=1,
```

```

        class_weight=inp.class_weight, # When none => ignored
    )

    return TrainOutput(
        model=inp.model,
        hist=hist)

```

✓ 5.1.2 Import & Export

Einfache Methoden zum Import und Export der Modelle. Erlaubt uns zusätzliche Infos mit anzuhängen.

```

from pathlib import Path
import json
import zipfile

def export_model(run: dict):
    out_dir: str = "export"
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    name = run["name"]

    model_path = out_dir / f"{name}.keras"
    meta_path = out_dir / f"{name}_meta.json"
    zip_path = out_dir / f"{name}_bundle.zip"

    # save model
    run["model"].save(model_path)

    # save meta
    meta = {
        "name": name,
        "model": run["model_name"],
        "hyperparams": {
            "lr": float(run["lr"]),
            "seed": int(run["seed"]),
            "epochs": int(run["epochs"]),
        }
    }
    meta_path.write_text(json.dumps(meta, indent=2), encoding="utf-8")

    # zip
    with zipfile.ZipFile(zip_path, "w", compression=zipfile.ZIP_DEFLATED) as zf:
        zf.write(model_path, arcname=model_path.name)
        zf.write(meta_path, arcname=meta_path.name)

    return str(zip_path)

```

```

from pathlib import Path
import zipfile
import tensorflow as tf

```



```

from tensorflow import keras
import gdown

def import_model(drive_url: str, bundle_name: str) -> tf.keras.Model:
    root = Path("import")
    root.mkdir(parents=True, exist_ok=True)

    zip_file = root / f"{bundle_name}.zip"
    extract_dir = root / bundle_name
    extract_dir.mkdir(parents=True, exist_ok=True)

    if (not zip_file.exists()) or (zip_file.stat().st_size == 0):
        gdown.download(
            url=drive_url,
            output=str(zip_file),
            quiet=False,
            fuzzy=True,
            use_cookies=True,
        )

    with zipfile.ZipFile(zip_file, "r") as zf:
        zf.extractall(extract_dir)

    candidates = list(extract_dir.rglob("*.keras"))
    model_path = candidates[0]
    return keras.models.load_model(model_path)

```

✓ 5.1.3 Resulttable

In den Experimenten werden wir öfters die Modelle über den ACC-Score hinaus evaluieren. Daher definieren wir uns im Vorfeld eine Methode, welche uns die Werte als Pandas DataFrame zurückgibt.

```

import numpy as np
import pandas as pd
from tensorflow import keras
from sklearn.metrics import (
    f1_score,
    precision_score,
    recall_score,
    accuracy_score,
    confusion_matrix,
)

def eval(model, data_dir: str, batch_size: int = 32, seed: int = 42, class_names=None) -> pd.DataFrame:
    ds = keras.utils.image_dataset_from_directory(
        data_dir,
        seed=seed,
        image_size=(180, 180),
        batch_size=batch_size,
        shuffle=False,

```

```

    class_names=class_names,
)

class_names = ds.class_names #[bottle, cable, ...]
C = len(class_names)
labels = np.arange(C)          #[0, 1, 2, 3, ...] <= Position of the categories

y_true = np.concatenate([y.numpy() for _, y in ds], axis=0).astype(int)      # Label values for all data e.g. [bottle_IO, bottle_IO, cable_IO, bottle_NIO ...]
probs = np.concatenate([model.predict(x, verbose=0) for x, _ in ds], axis=0)  # Probabilities as 3d array e.g. [ [0.8, 0.1, 0.1, ... ] (img1), [0.1, 0.7, 0.1, ...](img2) ]
y_pred = probs.argmax(axis=1).astype(int)                                     # Prediction. Prop flat to 1d by max in inner array e.g. [ 0, 1, ...]

prec_per_class = precision_score(y_true, y_pred, average=None, labels=labels, zero_division=0)
rec_per_class = recall_score(y_true, y_pred, average=None, labels=labels, zero_division=0)
f1_per_class = f1_score(y_true, y_pred, average=None, labels=labels, zero_division=0)

true_count = np.bincount(y_true, minlength=C)
pred_count = np.bincount(y_pred, minlength=C)

acc_per_class = np.array([
    accuracy_score((y_true == k).astype(int), (y_pred == k).astype(int))
    for k in range(C)
], dtype=float)

df = pd.DataFrame({
    "class": class_names,
    "true_count": true_count,
    "pred_count": pred_count,
    "precision": prec_per_class,
    "recall": rec_per_class,
    "f1": f1_per_class,
    "accuracy": acc_per_class,
}).sort_values("true_count", ascending=False).reset_index(drop=True)

summary = pd.DataFrame([
    {
        "class": "ALL",
        "true_count": int(len(y_true)),
        "pred_count": int(len(y_pred)),
        "precision": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
        "accuracy": float(accuracy_score(y_true, y_pred)),
    }
])

return pd.concat([df, summary], ignore_index=True)

```

✓ 5.1.4 Confusion Matrix

Stellt für ein Modell eine Confusion Matrix dar, indem es gegen das gegebene Datenverzeichnis prüft.

```

import numpy as np
from tensorflow import keras

```



```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

def plot_confusionmatrix(model, data_dir: str, batch_size: int = 32, seed: int = 42) -> None:
    ds = keras.utils.image_dataset_from_directory(
        data_dir,
        seed=seed,
        image_size=(180, 180),
        batch_size=batch_size,
        shuffle=False
    )

    class_names = ds.class_names  #[bottle, cable, ...]
    C = len(class_names)
    labels = np.arange(C)          #[0, 1, 2, 3, ...] <= Position of the categories

    y_true = np.concatenate([y.numpy() for _, y in ds], axis=0).astype(int)      # Label values for all data e.g. [bottle_IO, bottle_IO, cable_IO, bottle_NIO ...]
    probs = np.concatenate([model.predict(x, verbose=0) for x, _ in ds], axis=0)  # Probabilities as 3d array e.g. [ [0.8, 0.1, 0.1, ... ] (img1), [0.1, 0.7, 0.1, ...](img2) ]
    y_pred = probs.argmax(axis=1).astype(int)                                     # Prediction. Prop flat to 1d by max in inner array e.g. [ 0, 1, ...]

    cm = confusion_matrix(y_true, y_pred, normalize="true")

    fig, ax = plt.subplots(figsize=(22, 18))
    ConfusionMatrixDisplay(cm, display_labels=class_names).plot(
        ax=ax,
        cmap="Blues",
        xticks_rotation=45,
        values_format=".2f",
    )
    plt.show()

    return cm

```

✓ 5.2 Experiments - Custom Models

Diese Sektion zeigt unseren Learning-Pfad, den wir für die Custom-Modelle durchloffen sind.

Zusammenfassung des Lernipfads:

Als erstes reduzierten wir das Problem nur auf die Klassifikation von Kategorien. Um dieses Problem zu lösen probierten wir ein Custom VGG 16 aus. Dieses ist allerdings zu groß für diese Aufgabe. Daraufhin probierten wir ein kleineres Modell aus, ein Custom Xception. Auch dieses erwies sich als zu groß. Daraufhin probierten wir ein minimales CNN Modell aus. Dieses erreichte für die Klassifikation sehr gute Ergebnisse.

Das Modell haben wir anschließend für Kategorie-Klassifikation und IO/NIO-Klassifikation eingesetzt. Dies sah auf den ersten Moment sehr gut aus. Bei genauerer Betrachtung zeigte sich jedoch, dass das Set zu unbalanziert ist und das Modell eine hohe Accuracy erreicht, indem es die zahlenmäßig stark unterlegenen NIOs ignoriert. Um dieses Problem zu lösen haben wir 2 Maßnahmen ausprobiert. Zum Ersten die Klassengewichtung in der Lossfunktion, zum Zweiten ein Upsampling der NIOs durch Augmentierung. Beide Maßnahmen waren

sehr effektiv. Da die Augmentierung etwas bessere Ergebnisse brachte und beide Maßnahmen zusammen keinen Sinn ergeben, haben wir für die weitere Modellierung nur die augmentierten Daten verwendet.

Um das Modell weiter zu verbessern, haben wir für das Modell eine Hyperparametersuche durchgeführt. Wir haben dafür immer nur einen Parameter auf einmal ausgetauscht, um die Anzahl an notwendigen Trainings gering zu halten. Das getunte Modell hat auf dem Validierungsdatensatz einen besseren F1 Score als originale Modell. Uns ist beim Trainieren aufgefallen, dass die `val_loss` Metrik für Callbacks manchmal ungünstig ist. Dies haben wir anschließend auch unter der Verwendung des Tensorboards untersucht. Eine Confusion Matrix hat uns gezeigt, dass das Modell nur innerhalb einer Kategorie falsch vorhersagt.

Abschließend haben wir 2 Variationen des `simple_cnn` aufgebaut. Die erste Variation ist ein Ensemble an Classifiern. Erst die Kategorie und dann pro Kategorie ein eigener Classifier. Die Zweite Variation bildet ein einziger 2 Head-Classifier. In diesem Trainieren wir auf zwei Labels für zwei Heads und zwei Lossfunktionen.

Am Ende haben wir das getunte Modell, das Ensemble und den 2 Head-Classifier gegen die Testwerte ausgeführt und die Ergebnisse miteinander verglichen. Insgesamt haben sich alle 3 in ihrer Performance als vergleichbar herausgestellt.

Für das erstellen und trainieren aller Modelle gibt es ein paar grundlegende Überlegungen die wir uns gemacht haben.

Fehlerfunktion:

Wir verwenden Cross-Entropy als Loss-Funktion. Die Cross-Entropy misst, wie gut diese vorhergesagte Verteilung zur richtigen Klasse passt. Ist die vorhergesagte Wahrscheinlichkeit für die korrekte Klasse hoch, dann ist der Loss kleiner. Das erscheint uns für diese Aufgabe sinnvoll. Wir sind auf die Loss Funktion über das [Keras Tutorial](#) gekommen, in dem die binäre Version verwendet wird.

Implementierungstechnisch benötigen wir die `sparse_categorical_crossentropy` ([Link](#)), weil unsere Labels durch den Keras Loader mit `image_dataset_from_directory` als Integer-Klassenindizes vorliegen (e.g. 0 ... 29).

Damit wir ihn verwenden können, besitzen alle unsere Modelle einen Softmax-Output Layer, welcher die Ergebnisse so skaliert, dass das Ergebnis 1 ergibt. D.h. wir können den Output als Wahrscheinlichkeit pro Klasse interpretieren.

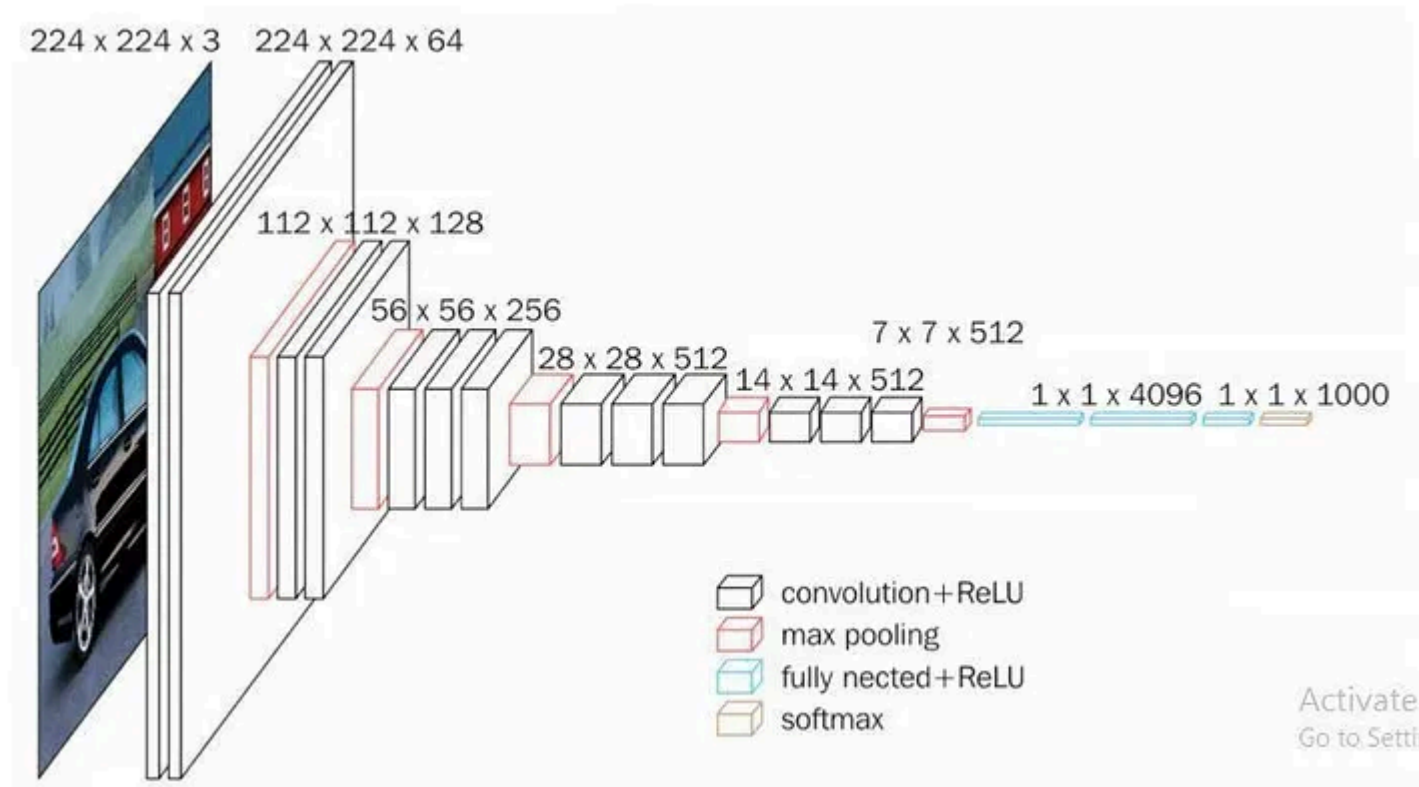
Optimizer:

Wir verwenden den in der Vorlesung vorgestellten Optimizer Adam.

✓ 5.2.1 Classify categories - VGG 16

In einem ersten Schritt haben wir das Problem erstmal nur auf die Kategorieklassifikation reduziert.

Für das Modell haben wir uns die in der Vorlesung vorgestellte VGG 16 Referenzarchitektur angeschaut und diese programmiert, auch wenn wir initial relativ sicher waren, dass dieses Modell für unsere einfache Klassifikation zu komplex ist und zu viele Parameter besitzt.



([Link](#))

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

def create_custom_vgg16(num_classes=15):
    # Pixels: 180x180, for each RGB Values
    input_shape=(180, 180, 3)

    inputs = keras.Input(shape=input_shape)      # Define input-layer
    x = layers.Resizing(224, 224)(inputs)         # The images are not 224x224. We use 180x180
    x = layers.Rescaling(1./255)(x)              # Rescaling RGB (reach from 0 - 255 divide by 255 to have a value within 0 - 1)

    # Meaning of: x = layers.Conv2D(filters=64, kernel_size=3, padding="same", activation="relu")(x)
    # Dim. automatically defined to 224x224 from the previous layer
    # Filters: 64 Filters
    # Kernel-size:3x3 window area
    # => Each Filter 3x3 Area multiplied by 3 values (RGB) => 27 Weights
    # Padding="same" => Keep 224x224x64. Otherwise it would be 222x222
    # (most left, right, top, bottom areas could not be calculated by 3x3 area)
    # Activation = "relu": Rectified linear units e.g. f(x) = max(0,x)

    # Feature learning
    x = layers.Conv2D(filters=64, kernel_size=3, padding="same", activation="relu")(x) # 224x224
    x = layers.Conv2D(filters=64, kernel_size=3, padding="same", activation="relu")(x)

    x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x) # 112x112 (Biggest value of 2x2 area. Strides 2 => Stepsize2 => Dim/2)
    x = layers.Conv2D(filters=128, kernel_size=3, padding="same", activation="relu")(x)
    x = layers.Conv2D(filters=128, kernel_size=3, padding="same", activation="relu")(x)
```

```

x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x) # 56x56
x = layers.Conv2D(filters=256, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=256, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=256, kernel_size=3, padding="same", activation="relu")(x)

x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x) # 28x28
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)

x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x) # 14x14
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)
x = layers.Conv2D(filters=512, kernel_size=3, padding="same", activation="relu")(x)

# Classification
x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x) # 7x7
x = layers.Flatten()(x) # 7x7 (area) x 512 Filter => 25088
x = layers.Dense(4096, activation="relu")(x) # 25088 => 4096
x = layers.Dense(4096, activation="relu")(x)
x = layers.Dense(4096, activation="relu")(x)
outputs = layers.Dense(num_classes, activation="softmax")(x) # Prediction layer => Probabilities

return keras.Model(inputs, outputs, name="vgg16_custom")

```

```

from tensorflow import keras

lr = 1e-3
epochs = 30
seed = 42

model = create_custom_vgg16()
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_not_augmented_split_categories_only",
    model=model,
    seed=seed,
    epochs=epochs,
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,

```

```
epochs=epochs
)

zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model("https://drive.google.com/file/d/1j2nVBuVl-2qeMc1PP05WhApJouHitkJR/view?usp=sharing", "vgg16_custom_bundle")
```

```
df = eval(model, "data/data_not_augmented_split_categories_only/val", batch_size=32, seed=42)
df
```

Found 534 files belonging to 15 classes.

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut	51	534	0.095506	1.000000	0.174359	0.095506
1	screw	46	0	0.000000	0.000000	0.000000	0.913858
2	pill	43	0	0.000000	0.000000	0.000000	0.919476
3	carpet	41	0	0.000000	0.000000	0.000000	0.923221
4	zipper	41	0	0.000000	0.000000	0.000000	0.923221
5	leather	38	0	0.000000	0.000000	0.000000	0.928839
6	cable	36	0	0.000000	0.000000	0.000000	0.932584
7	tile	36	0	0.000000	0.000000	0.000000	0.932584
8	capsule	34	0	0.000000	0.000000	0.000000	0.936330
9	wood	33	0	0.000000	0.000000	0.000000	0.938202
10	grid	33	0	0.000000	0.000000	0.000000	0.938202
11	metal_nut	32	0	0.000000	0.000000	0.000000	0.940075
12	transistor	31	0	0.000000	0.000000	0.000000	0.941948
13	bottle	29	0	0.000000	0.000000	0.000000	0.945693
14	toothbrush	10	0	0.000000	0.000000	0.000000	0.981273
15	ALL	534	534	0.006367	0.066667	0.011624	0.095506

Der Trainingsverlauf zeigt wie vermutet, das dieses Modell viel zu komplex ist bzw. massiv überdimensioniert für dieses Problem/den Datensatz ist. Es tritt kein Lern-Effekt ein. Es wird nur die Haselnut vorhergesagt. Aufgrund desssen haben wir in dem nachfolgenden Versuch das Modell deutlich leichter gemacht.

✓ 5.2.2 Classify categories - Xception

Xception Modell aus dem Keras Guide [Image classification from scratch](#)

```
def make_xception(input_shape, num_classes):
    inputs = keras.Input(shape=input_shape)
```

```

# Entry block
x = layers.Rescaling(1.0 / 255)(inputs)
x = layers.Conv2D(128, 3, strides=2, padding="same")(x)
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x)

previous_block_activation = x # Set aside residual

for size in [256, 512, 728]:
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same")(x)
    x = layers.BatchNormalization()(x)

    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same")(x)
    x = layers.BatchNormalization()(x)

    x = layers.MaxPooling2D(3, strides=2, padding="same")(x)

    # Project residual
    residual = layers.Conv2D(size, 1, strides=2, padding="same")(
        previous_block_activation
    )
    x = layers.add([x, residual]) # Add back residual
    previous_block_activation = x # Set aside next residual

x = layers.SeparableConv2D(1024, 3, padding="same")(x)
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x)

x = layers.GlobalAveragePooling2D()(x)
if num_classes == 2:
    units = 1
else:
    units = num_classes

x = layers.Dropout(0.25)(x)
# We specify activation=None so as to return logits
outputs = layers.Dense(units, activation=None)(x)
return keras.Model(inputs, outputs, name="xception")

```

```
from tensorflow import keras
```

```

lr = 1e-3
epochs = 30
seed = 42
batch_size = 32
image_size = (180, 180)

```

```
model = make_xception(input_shape=image_size + (3,), num_classes=15)
```

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr),

```



```
        loss= keras.losses.SparseCategoricalCrossentropy(),
        metrics=["accuracy"],
    )

    inp = TrainInput(
        data_dir="data/data_not_augmented_split_categories_only",
        model=model,
        batch_size=batch_size,
        seed=seed,
        epochs=epochs,
    )

    out = train(inp)

    run = dict(
        name=out.model.name,
        model=out.model,
        model_name=out.model.name,
        lr=lr,
        seed=seed,
        epochs=epochs
    )

    zip_path = export_model(run)
    print("Exported:", zip_path)

df = eval(out.model, "data/data_not_augmented_split_categories_only/test", batch_size=batch_size, seed=seed)
df
```

► Training-Log

```
model = import_model("https://drive.google.com/file/d/1pw8pnxt7xGlnAAtVrx4hUlqg1IOaFy-h/view?usp=sharing", "xception_bundle")
```

```
df = eval(model, "data/data_not_augmented_split_categories_only/val", batch_size=32, seed=42)
df
```

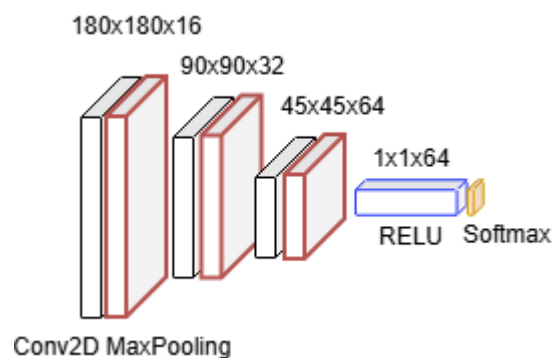

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut	51	0	0.000000	0.000000	0.000000	0.904494
1	screw	46	138	0.000000	0.000000	0.000000	0.655431
2	pill	43	0	0.000000	0.000000	0.000000	0.919476
3	carpet	41	0	0.000000	0.000000	0.000000	0.923221
4	zipper	41	0	0.000000	0.000000	0.000000	0.923221
5	leather	38	0	0.000000	0.000000	0.000000	0.928839
6	cable	36	0	0.000000	0.000000	0.000000	0.932584
7	tile	36	0	0.000000	0.000000	0.000000	0.932584
8	capsule	34	0	0.000000	0.000000	0.000000	0.936330
9	wood	33	396	0.083333	1.000000	0.153846	0.320225
10	grid	33	0	0.000000	0.000000	0.000000	0.938202
11	metal_nut	32	0	0.000000	0.000000	0.000000	0.940075
12	transistor	31	0	0.000000	0.000000	0.000000	0.941948
13	bottle	29	0	0.000000	0.000000	0.000000	0.945693
14	toothbrush	10	0	0.000000	0.000000	0.000000	0.981273
15	ALL	534	534	0.005556	0.066667	0.010256	0.061798

Der Trainingsverlauf zeigt auch hier, dass dieses Modell zu komplex ist bzw. zu überdimensioniert für dieses Problem/den Datensatz ist. Auch hier tritt praktisch kein Lern-Effekt ein. An der gleichen `val_accuracy` und `val_loss` lässt sich vermuten, dass das Modell schon sehr schnell anfängt nur wenige Klassen vorherzusagen. Dies bestätigt die Ergebnistabelle. Es werden nur 2 Klassen vorhergesagt. Im nächsten Versuch haben wir daher eine Architektur gewählt, welche Minimal ist.

✓ 5.2.3 Classifiy categories - Custom simple cnn

Modellaufbau:

Die Modellarchitektur ist stark an das VGG16 angelehnt, jedoch absolut minimal gehalten. Es gibt nur 3 Convolutional-MaxPooling-Layers, welche als Backbone dienen. Der Classification-Head besteht aus einem einzelnen Dense Layer mit Rectified Linear Aktivierungsfunktion. Diesen haben wir später um einen Dropout erweitert, da er sich sehr wirksam gegen Overfitting erwiesen hat. Den Output-Layer bildet die Softmax-Funktion.




```
def make_simple_cnn_category_only(input_shape=(180, 180, 3), num_classes=15):
    inputs = keras.Input(shape=input_shape)

    x = layers.Rescaling(1.0/255)(inputs)

    x = layers.Conv2D(16, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Flatten()(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="simple_cnn_category_only")
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_simple_cnn_category_only()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_not_augmented_split_categories_only",
    model=model,
    seed=seed,
    epochs=epochs,
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs
)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Trainings-Log

```
model = import_model(
    "https://drive.google.com/file/d/1B94yhHx2SJVVcEqCEdUBrJwt0q4Mn0Rf/view?usp=sharing",
    "simple_cnn_category_only_bundle")
```

```
df = eval(model, "data/data_not_augmented_split_categories_only/val", batch_size=32, seed=42)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut	51	51	1.0	1.0	1.0	1.0
1	screw	46	46	1.0	1.0	1.0	1.0
2	pill	43	43	1.0	1.0	1.0	1.0
3	carpet	41	41	1.0	1.0	1.0	1.0
4	zipper	41	41	1.0	1.0	1.0	1.0
5	leather	38	38	1.0	1.0	1.0	1.0
6	cable	36	36	1.0	1.0	1.0	1.0
7	tile	36	36	1.0	1.0	1.0	1.0
8	capsule	34	34	1.0	1.0	1.0	1.0
9	wood	33	33	1.0	1.0	1.0	1.0
10	grid	33	33	1.0	1.0	1.0	1.0
11	metal_nut	32	32	1.0	1.0	1.0	1.0
12	transistor	31	31	1.0	1.0	1.0	1.0
13	bottle	29	29	1.0	1.0	1.0	1.0
14	toothbrush	10	10	1.0	1.0	1.0	1.0
15	ALL	534	534	1.0	1.0	1.0	1.0

Wir sind davon ausgegangen, das die Kategorieaufgabe recht schnell gelernt werden kann, da an sehr einfachen Eigenschaften (z.B. die Hintergrundfarbe) gelernt werden kann.

Das Ergebnis war dennoch unerwartet. Der Trainingsverlauf zeigte, dass das Modell die Kategorien sehr schnell lernt ohne dabei ein Overfitt zu verursachen. Das Validationergebnis zeigt, das die Klassifikation der Kategorien perfekt ist.

Aufgrund dessen bearbeiten wir für die Kategorieklassifikation dieses Modell nicht weiter und prüfen es gegen die Evaluationswerte.

```
df = eval(model, "data/data_not_augmented_split_categories_only/test", batch_size=32, seed=42)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw	51	51	1.0	1.0	1.0	1.0
1	hazelnut	49	49	1.0	1.0	1.0	1.0
2	pill	44	44	1.0	1.0	1.0	1.0
3	carpet	38	38	1.0	1.0	1.0	1.0
4	cable	38	38	1.0	1.0	1.0	1.0
5	leather	36	36	1.0	1.0	1.0	1.0
6	capsule	36	36	1.0	1.0	1.0	1.0
7	zipper	36	36	1.0	1.0	1.0	1.0
8	metal_nut	35	35	1.0	1.0	1.0	1.0
9	tile	34	34	1.0	1.0	1.0	1.0
10	grid	34	34	1.0	1.0	1.0	1.0
11	wood	32	32	1.0	1.0	1.0	1.0
12	transistor	32	32	1.0	1.0	1.0	1.0
13	bottle	29	29	1.0	1.0	1.0	1.0
14	toothbrush	10	10	1.0	1.0	1.0	1.0
15	ALL	534	534	1.0	1.0	1.0	1.0

Die Evaluationswerte bestätigen die Vermutung.

✓ 5.2.4 Classify (categories x io/nio)

Da die Klassifikation der Kategorien so gut funktioniert, wenden wir das Modell direkt auf Gesamtproblem an, indem wir jede Kategorie in IO und NIO klassifizieren. Dadurch ergibt sich ein 30er Klassifikationsproblem.



```
def make_simple_cnn_category_and_io_nio(input_shape=(180, 180, 3), num_classes=30):  
    inputs = keras.Input(shape=input_shape)  
  
    x = layers.Rescaling(1.0/255)(inputs)  
  
    x = layers.Conv2D(16, 3, padding="same", activation="relu")(x)  
    x = layers.MaxPooling2D()(x)  
  
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)  
    x = layers.MaxPooling2D()(x)  
  
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)  
    x = layers.MaxPooling2D()(x)  
  
    x = layers.Flatten()(x)  
    x = layers.Dense(64, activation="relu")(x)  
    x = layers.Dropout(0.3)(x)  
  
    outputs = layers.Dense(num_classes, activation="softmax")(x)  
    return keras.Model(inputs, outputs, name="simple_cnn_category_and_io_nio")
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_simple_cnn_category_and_io_nio()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_not_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs
)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Trainings-Log

```
model = import_model(
    "https://drive.google.com/file/d/1kRtMHQm04SZjJ9sIe_Ar_cocY0M6N1aH/view?usp=sharing",
    "simple_cnn_category_and_io_nio")
```

```
df = eval(model, "data/data_not_augmented_split/val", batch_size=32, seed=42)
df
```


Found 534 files belonging to 30 classes.

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_IO	43	50	0.860000	1.000000	0.924731	0.986891
1	screw_IO	36	46	0.782609	1.000000	0.878049	0.981273
2	carpet_IO	31	41	0.756098	1.000000	0.861111	0.981273
3	pill_IO	29	43	0.674419	1.000000	0.805556	0.973783
4	grid_IO	28	33	0.848485	1.000000	0.918033	0.990637
5	leather_IO	28	36	0.777778	1.000000	0.875000	0.985019
6	cable_IO	28	31	0.806452	0.892857	0.847458	0.983146
7	wood_IO	27	33	0.818182	1.000000	0.900000	0.988764
8	transistor_IO	27	30	0.900000	1.000000	0.947368	0.994382
9	zipper_IO	27	41	0.658537	1.000000	0.794118	0.973783
10	tile_IO	26	33	0.757576	0.961538	0.847458	0.983146
11	metal_nut_IO	24	30	0.800000	1.000000	0.888889	0.988764
12	capsule_IO	24	34	0.705882	1.000000	0.827586	0.981273
13	bottle_IO	23	27	0.851852	1.000000	0.920000	0.992509
14	zipper_NIO	14	0	0.000000	0.000000	0.000000	0.973783
15	pill_NIO	14	0	0.000000	0.000000	0.000000	0.973783
16	tile_NIO	10	3	0.666667	0.200000	0.307692	0.983146
17	screw_NIO	10	0	0.000000	0.000000	0.000000	0.981273
18	carpet_NIO	10	0	0.000000	0.000000	0.000000	0.981273
19	capsule_NIO	10	0	0.000000	0.000000	0.000000	0.981273
20	leather_NIO	10	2	1.000000	0.200000	0.333333	0.985019
21	hazelnut_NIO	8	1	1.000000	0.125000	0.222222	0.986891
22	cable_NIO	8	5	0.400000	0.250000	0.307692	0.983146
23	metal_nut_NIO	8	2	1.000000	0.250000	0.400000	0.988764
24	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.994382
25	bottle_NIO	6	2	1.000000	0.333333	0.500000	0.992509
26	wood_NIO	6	0	0.000000	0.000000	0.000000	0.988764
27	grid_NIO	5	0	0.000000	0.000000	0.000000	0.990637
28	transistor_NIO	4	1	1.000000	0.250000	0.400000	0.994382
29	toothbrush_NIO	3	0	0.000000	0.000000	0.000000	0.994382
30	ALL	534	534	0.592151	0.548758	0.517661	0.779026

Das Modell hat sich als verwenbar erwiesen und scheint zunächst gute Ergebnisse zu liefern. Jedoch **Voricht** ⚠️. Die **Accuracy täuscht**, aufgrund der starken Class Imbalance zwischen IO und NIO.

In der Tabelle lässt sich dies sehr gut an den PredCounts, Precision, Recall und vorallem dem F1 Scores der NIO Klassen ablesen. Im folgenden nutzen wir daher den **F1-Score** als Vergleichsmetrik, weil er die Precision und den Recall zusammenfasst und damit bei der Class Imbalance aussagekräftiger ist als (unbalanced) Accuracy.

Insgesamt gibt es einige Maßnahmen die wir treffen können um dies zu reduzieren. So kann z.B. Downsampling eingesetzt werden. D.h. man verwendet nicht alle Bilder welche in Ordnung sind um das Klassenungleichgewicht abzuschwächen. In unseren Fall haben wir uns dagegen entschieden, da der Datensatz bereits sehr wenige Daten beinhaltet. Im Folgenden verfolgen wir daher zwei andere Maßnahmen um die Imbalance anzupassen.

Einmal über die Anpassung Loss-Funktion und einmal in dem wir die NIO Bilder augmentieren. Wir halten uns dabei relativ nah an dem [TensorFlow Guide](#).

Im ersten Fall müssen wir die Daten nicht verändern. Stattdessen trimmen wir das Modell dazu auf NIO vorherzusagen, in dem wir eine Fehlklassifikation eines NIO Elements deutlich stärker - abhängig von der Anzahl wie oft das Element im Verhältnis zu IO vorkommt - gewichten. In der zweiten Maßnahme erhöhen wir den Anteil an NIO Bilder, indem wir diese durch Data Augmentation (aus der Data Preparation Phase) upsampeln.

✓ 5.2.5 Classify (categories x io/nio) with class weights

Wir verwenden die `compute_class_weights` und übergeben sie der Training-Pipeline. Diese werden dem kompilierten Modell beim `fit` übergeben.

Die Klassengewichte ermitteln wir anhand des Trainingsdatensatzes unter der Verwendung von SkLearns [compute_class_weights](#). Hierbei wird das Klassenhäufigkeitsverhältnis miteinberechnet. Gibt es doppelt so viele IO Samples einer Kategorie gegenüber der NIO Samples. So werden die NIO Samples gegenüber den IO Samples doppelt so stark gewichtet.

```
def make_simple_cnn_category_and_io_nio_class_weights(input_shape=(180, 180, 3), num_classes=30):
    inputs = keras.Input(shape=input_shape)

    x = layers.Rescaling(1.0/255)(inputs)

    x = layers.Conv2D(16, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Flatten()(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="simple_cnn_category_and_io_nio_class_weights")
```

```
import numpy as np
from tensorflow import keras
from sklearn.utils.class_weight import compute_class_weight

def compute_class_weights(train_dir: str, seed: int, image_size=(180, 180), batch_size=32):
```



```

ds = keras.utils.image_dataset_from_directory(
    train_dir,
    seed=seed,
    image_size=image_size,
    batch_size=batch_size,
    shuffle=False,
)
y = np.concatenate([yy.numpy() for _, yy in ds], axis=0).astype(int)
classes = np.unique(y)
w = compute_class_weight(class_weight="balanced", classes=classes, y=y)
return {int(c): float(w) for c, w in zip(classes, w)}

```

```

import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_simple_cnn_category_and_io_nio_class_weights()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

class_weight = compute_class_weights(
    "data/data_not_augmented_split/train",
    seed=seed
)

inp = TrainInput(
    data_dir="data/data_not_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    class_weight=class_weight
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs
)
zip_path = export_model(run)
print("Exported:", zip_path)

```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/19i8kQ6GK1iREX-WVHFd8FbaFiZQRlhTS/view?usp=sharing",
    "simple_cnn_category_and_io_nio_class_weights_bundle")
```

```
df = eval(model, "data/data_not_augmented_split/val", batch_size=32, seed=42)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_IO	43	44	0.954545	0.976744	0.965517	0.994382
1	screw_IO	36	35	0.800000	0.777778	0.788732	0.971910
2	carpet_IO	31	27	0.814815	0.709677	0.758621	0.973783
3	pill_IO	29	15	0.866667	0.448276	0.590909	0.966292
4	grid_IO	28	29	0.862069	0.892857	0.877193	0.986891
5	leather_IO	28	24	0.958333	0.821429	0.884615	0.988764
6	cable_IO	28	28	0.857143	0.857143	0.857143	0.985019
7	wood_IO	27	18	0.888889	0.592593	0.711111	0.975655
8	transistor_IO	27	29	0.931034	1.000000	0.964286	0.996255
9	zipper_IO	27	28	0.892857	0.925926	0.909091	0.990637
10	tile_IO	26	31	0.806452	0.961538	0.877193	0.986891
11	metal_nut_IO	24	26	0.846154	0.916667	0.880000	0.988764
12	capsule_IO	24	26	0.807692	0.875000	0.840000	0.985019
13	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998127
14	zipper_NIO	14	13	0.846154	0.785714	0.814815	0.990637
15	pill_NIO	14	28	0.428571	0.857143	0.571429	0.966292
16	tile_NIO	10	4	0.750000	0.300000	0.428571	0.985019
17	screw_NIO	10	11	0.272727	0.300000	0.285714	0.971910
18	carpet_NIO	10	14	0.357143	0.500000	0.416667	0.973783
19	capsule_NIO	10	8	0.625000	0.500000	0.555556	0.985019
20	leather_NIO	10	14	0.642857	0.900000	0.750000	0.988764
21	hazelnut_NIO	8	7	0.857143	0.750000	0.800000	0.994382
22	cable_NIO	8	8	0.500000	0.500000	0.500000	0.985019
23	metal_nut_NIO	8	6	0.666667	0.500000	0.571429	0.988764
24	toothbrush_IO	7	7	0.857143	0.857143	0.857143	0.996255
25	bottle_NIO	6	5	1.000000	0.833333	0.909091	0.998127
26	wood_NIO	6	15	0.266667	0.666667	0.380952	0.975655
27	grid_NIO	5	5	0.400000	0.400000	0.400000	0.988764
28	transistor_NIO	4	2	1.000000	0.500000	0.666667	0.996255
29	toothbrush_NIO	3	3	0.666667	0.666667	0.666667	0.996255
30	ALL	534	534	0.746057	0.719076	0.715261	0.784644

Man sieht, dass durch diese Änderung das Modell die NIOs nicht mehr so stark vernachlässigt. Der F1 Score von 0.51 auf 0.71 stark angestiegen ist. Die Pred-Counts bestätigen dies ebenfalls.

✓ 5.2.6 Classify (categories x io/nio) with augmentation

Erklären Idee wir augmentieren die NIO bilder um imbalance anzupassen

```
def make_simple_cnn_with_augmentation(input_shape=(180, 180, 3), num_classes=30):
    inputs = keras.Input(shape=input_shape)

    x = layers.Rescaling(1.0/255)(inputs)

    x = layers.Conv2D(16, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Flatten()(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="simple_cnn_with_augmentation")
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_simple_cnn_with_augmentation()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
)

out = train(inp)
```

```
run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs
)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/1Kzyk1hd0xrIjUUonZhp9796dC_kkWxs-/view?usp=sharing",
    "simple_cnn_category_and_io_nio_class_weights_bundle")
```

```
df = eval(model, "data/data_augmented_split/val", batch_size=32, seed=43)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	36	1.000000	0.720000	0.837209	0.982948
1	hazelnut_IO	43	57	0.754386	1.000000	0.860000	0.982948
2	screw_IO	36	44	0.659091	0.805556	0.725000	0.973203
3	carpet_NIO	35	35	0.742857	0.742857	0.742857	0.978076
4	tile_NIO	32	25	1.000000	0.781250	0.877193	0.991474
5	carpet_IO	31	31	0.709677	0.709677	0.709677	0.978076
6	zipper_NIO	31	29	0.931034	0.870968	0.900000	0.992692
7	screw_NIO	31	23	0.695652	0.516129	0.592593	0.973203
8	leather_NIO	30	32	0.843750	0.900000	0.870968	0.990256
9	pill_IO	29	15	0.866667	0.448276	0.590909	0.978076
10	leather_IO	28	26	0.884615	0.821429	0.851852	0.990256
11	grid_IO	28	53	0.509434	0.964286	0.666667	0.967113
12	pill_NIO	28	42	0.619048	0.928571	0.742857	0.978076
13	cable_IO	28	29	0.896552	0.928571	0.912281	0.993910
14	wood_NIO	27	27	0.592593	0.592593	0.592593	0.973203
15	zipper_IO	27	29	0.862069	0.925926	0.892857	0.992692
16	transistor_IO	27	38	0.710526	1.000000	0.830769	0.986602
17	wood_IO	27	27	0.592593	0.592593	0.592593	0.973203
18	transistor_NIO	27	16	1.000000	0.592593	0.744186	0.986602
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	4	0.750000	0.120000	0.206897	0.971985
21	cable_NIO	25	24	0.916667	0.880000	0.897959	0.993910
22	metal_nut_IO	24	27	0.814815	0.916667	0.862745	0.991474
23	capsule_IO	24	34	0.647059	0.916667	0.758621	0.982948
24	bottle_IO	23	28	0.821429	1.000000	0.901961	0.993910
25	capsule_NIO	22	12	0.833333	0.454545	0.588235	0.982948
26	bottle_NIO	22	17	1.000000	0.772727	0.871795	0.993910
27	metal_nut_NIO	22	19	0.894737	0.772727	0.829268	0.991474
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.998782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.810671	0.783598	0.774614	0.779537

Man sieht auch hier, dass durch diese Änderung das Modell die NIOs nicht mehr so stark vernachlässigt. Der F1 Score ist hier von 0.51 auf 0.77 stark angestiegen. Auch hier bestätigen die Pred-Counts dies, wobei bei diesen beachtet werden muss, dass hier nun auch der true_count durch die Augmentierung höher ist.

✓ 5.2.7 Compare class weights vs augmentation

Werden die beiden Validierungstabellen betrachtet, so sehen wir das beider Augmentierung das Modell insgesamt etwas besser abgeschnitten hat. Knapp 6%. Dieser Abstand bleibt auch ungefähr wenn wir verschiedene Seeds verwenden. Wir haben uns deshalb dazu entschieden, mit der Augmentierung weiter zu arbeiten.

Beim ausführen mit verschiedenen Seeds haben wir bereits bemerkt, das dieses Modell - bedingt durch die geringe Anzahl an originalen NIO Bildern - nicht sehr stabil ist.

✓ 5.2.8 Tuning simple cnn

Ursprünglich wollten wir das Modell mittels eines Keras Autotuners trainieren. Wir haben uns dann jedoch dagegen entschieden, da wir zu viele Parameter ausprobieren wollten und eine sehr hohe Anzahl an Trainings gebraucht hätten. Daher haben wir eine Basiskonfiguration ausgewählt (so wie bisher) und dann immer einen Parameter angepasst. Wir erhoffen uns davon, effizient herauszufinden, welche Parameter einen Einfluss haben und in welche Richtung wir diese anpassen müssen.

Um die Trainingszeit klein zu halten, haben wir zudem die Anzahl an Epochen auf 15 beschränkt.

```
from tensorflow import keras
from keras import layers
import pandas as pd

def make_cnn_from_cfg(cfg, num_classes=30):
    inputs = keras.Input(shape=(180,180,3))
    x = layers.Rescaling(1./255)(inputs)

    filters_map = {
        "small": [16, 32, 64, 128],
        "medium": [24, 48, 96, 192],
        "large": [32, 64, 128, 256],
    }
    filters_per_block = filters_map[cfg["filters"]]

    for f in filters_per_block[:cfg["blocks"]]:
        x = layers.Conv2D(f, 3, padding="same", activation="relu")(x)
        x = layers.MaxPooling2D()(x)

    if cfg["use_gap"]:
        x = layers.GlobalAveragePooling2D()(x)
    else:
        x = layers.Flatten()(x)

    for _ in range(cfg["dense_layers"]):
        x = layers.Dense(cfg["dense_units"], activation="relu")(x)

    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name=cfg["name"])
    model.compile(
        optimizer=keras.optimizers.Adam(cfg["lr"]),
        loss="sparse_categorical_crossentropy",
```



```

        metrics=["accuracy"],
    )
    return model

```

```

BASE = dict(
    blocks=3,
    filters="medium",
    use_gap=False,
    dense_layers=1,
    dense_units=64,
    lr=3e-4
)

runs = [
    dict(name="baseline", **BASE),

    dict(name="blocks_2", **{**BASE, "blocks": 2}),
    dict(name="blocks_4", **{**BASE, "blocks": 4}),

    dict(name="filters_small", **{**BASE, "filters": "small"}),
    dict(name="filters_large", **{**BASE, "filters": "large"}),

    dict(name="use_gap", **{**BASE, "use_gap": True}),
    dict(name="dense_layers_2", **{**BASE, "dense_layers": 2}),
    dict(name="dense_128", **{**BASE, "dense_units": 128}),
]

```

```

import pandas as pd
import keras

data_dir = "data/data_augmented_split"
seed = 44
epochs = 20

results = []

for cfg in runs:
    print("\n=====")
    print("RUN:", cfg["name"])
    print(cfg)
    print("=====")

    model = make_cnn_from_cfg(cfg)

    model.compile(
        optimizer=keras.optimizers.Adam(cfg["lr"]),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )

    inp = TrainInput(

```

```

        data_dir=data_dir,
        model=model,
        seed=seed,
        epochs=epochs,
        batch_size=32,
    )

    out = train(inp)

    df_val = eval(out.model, f"{data_dir}/val", batch_size=32, seed=seed)
    val_acc = float(df_val.loc[df_val["class"] == "ALL", "accuracy"].iloc[0])
    val_f1 = float(df_val.loc[df_val["class"] == "ALL", "f1"].iloc[0])

    results.append({
        "name": cfg["name"],
        "blocks": cfg["blocks"],
        "filters": cfg["filters"],
        "use_gap": cfg["use_gap"],
        "dense_layers": cfg["dense_layers"],
        "dense_units": cfg["dense_units"],
        "lr": cfg["lr"],
        "accuracy": val_acc,
        "f1": val_f1
    })

res_df = pd.DataFrame(results).sort_values("f1", ascending=False).reset_index(drop=True)
res_df

```

	name	blocks	filters	use_gap	dense_layers	dense_units	lr	accuracy	f1
0	blocks_4	4	medium	False	1	64	0.0003	0.803898	0.799060
1	dense_128	3	medium	False	1	128	0.0003	0.808770	0.796494
2	baseline	3	medium	False	1	64	0.0003	0.778319	0.768647
3	blocks_2	2	medium	False	1	64	0.0003	0.773447	0.756934
4	dense_layers_2	3	medium	False	2	64	0.0003	0.755177	0.737358
5	filters_small	3	small	False	1	64	0.0003	0.750305	0.733872
6	filters_large	3	large	False	1	64	0.0003	0.728380	0.704806
7	use_gap	3	medium	True	1	64	0.0003	0.641900	0.535766

► Tuning-Log

Als Tuningergebnis haben sich folgende Parameter vereinzelt als vorteilhaft gegenüber dem Basismodell erwiesen.

- 4 Conv2D-MaxPooling2D Blocks
- Bestehende Filtergröße
- 1 Dense Layer
- Dense Layer mit 128 Blocks
- Flatten statt Global Average Pooling

Uns ist bewusst, dass wenn wir alle Parameter, die vereinzelt einen positiven Effekt haben, anwenden, wir nicht automatisch das beste Modelle erhalten. Um jedoch die beste Kombination herauszufinden, wären viele Trainingseinheiten notwendig. Im nächsten Schritt haben wir uns daher entschieden all diese Parameter in dem Modell anzuwenden.

✓ 5.2.9 Run tuned simple cnn

Im Folgenden wird das simple_cnn Modell mit den ermittelten Parametern trainiert.

```
from tensorflow import keras
from keras import layers

def simple_cnn_tuned(input_shape=(180, 180, 3), num_classes=30, lr=3e-4):
    inputs = keras.Input(shape=input_shape)
    x = layers.Rescaling(1./255)(inputs)

    for f in [24, 48, 96, 192]:
        x = layers.Conv2D(f, 3, padding="same", activation="relu")(x)
        x = layers.MaxPooling2D()(x)

    x = layers.Flatten()(x)

    x = layers.Dense(64, activation="relu")(x)

    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(num_classes, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name="simple_cnn_tuned")
    model.compile(
        optimizer=keras.optimizers.Adam(lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = simple_cnn_tuned()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
```

```
data_dir="data/data_augmented_split",
model=model,
seed=seed,
epochs=epochs,
tensorboard_logdir="logs/simple_cnn_tuned",
callback_metric="val_accuracy"
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs,
)

zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/11dIeq5jv0vJ0tj80eWbqwk0u1JKlM5AL/view?usp=sharing",
    "simple_cnn_tuned_bundle")
```

```
df = eval(model, "data/data_augmented_split/val", batch_size=32, seed=43)
df
```

Found 821 files belonging to 30 classes.

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	44	0.977273	0.860000	0.914894	0.990256
1	hazelnut_IO	43	49	0.857143	0.976744	0.913043	0.990256
2	screw_IO	36	29	0.620690	0.500000	0.553846	0.964677
3	carpet_NIO	35	36	0.777778	0.800000	0.788732	0.981730
4	tile_NIO	32	11	0.909091	0.312500	0.465116	0.971985
5	carpet_IO	31	30	0.766667	0.741935	0.754098	0.981730
6	zipper_NIO	31	28	0.964286	0.870968	0.915254	0.993910
7	screw_NIO	31	38	0.526316	0.645161	0.579710	0.964677
8	leather_NIO	30	29	0.931034	0.900000	0.915254	0.993910
9	pill_IO	29	40	0.675000	0.931034	0.782609	0.981730
10	leather_IO	28	29	0.896552	0.928571	0.912281	0.993910
11	grid_IO	28	28	0.821429	0.821429	0.821429	0.987820
12	pill_NIO	28	17	0.882353	0.535714	0.666667	0.981730
13	cable_IO	28	34	0.705882	0.857143	0.774194	0.982948
14	wood_NIO	27	21	0.952381	0.740741	0.833333	0.990256
15	zipper_IO	27	30	0.866667	0.962963	0.912281	0.993910
16	transistor_IO	27	30	0.900000	1.000000	0.947368	0.996346
17	wood_IO	27	33	0.787879	0.962963	0.866667	0.990256
18	transistor_NIO	27	24	1.000000	0.888889	0.941176	0.996346
19	tile_IO	26	43	0.581395	0.961538	0.724638	0.976857
20	grid_NIO	25	29	0.689655	0.800000	0.740741	0.982948
21	cable_NIO	25	19	0.789474	0.600000	0.681818	0.982948
22	metal_nut_IO	24	28	0.821429	0.958333	0.884615	0.992692
23	capsule_IO	24	31	0.741935	0.958333	0.836364	0.989038
24	bottle_IO	23	27	0.851852	1.000000	0.920000	0.995128
25	capsule_NIO	22	15	0.933333	0.636364	0.756757	0.989038
26	bottle_NIO	22	18	1.000000	0.818182	0.900000	0.995128
27	metal_nut_NIO	22	18	0.944444	0.772727	0.850000	0.992692
28	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996346
29	toothbrush_NIO	6	3	1.000000	0.500000	0.666667	0.996346
30	ALL	821	821	0.829065	0.808074	0.801436	0.808770

Das getune Modell hat bessere Werte als das Originalmodell. Der F1 Score ist minimal höher als in der besten Konfiguration aus 5.2.8. Das zeigt, dass die Auswahl beider Hyperparameter die richtige Entscheidung war.

Über mehrere Trainingsläufe hinweg haben wir beobachtet, dass das Modell nicht ganz stabil ist. In einzelnen Runs bricht die Performance ein und es stellt sich nur ein F1-Score um ca. 0.7 ein. Dieses Verhalten trat genau dann auf, wenn in den Callback-Funktionen `val_loss` als Monitoring-Metrik verwendet wurde und das Training anschließend ein Modell aus einer frühen Epoche wiederherstellte. In solchen Fällen kam es zu Verwechslungen, bei denen für einzelne Produktkategorien die IO/NIO-Klassen vertauscht wurden.

Dadurch das die `val_loss` Metrik (wir verwenden Cross-Entropy) nicht nur "richtig/falsch" bewertet, sondern auch die Sicherheit der Vorhersagen, könnte es sein, das einzelne überkonfidente Fehlklassifikationen den Loss so stark erhöhen, dass obwohl spätere Epochen bessere Klassifikationsleistung (und in unserem Fall häufig auch besseren F1-Score) liefern, sich nicht wieder (schnell genug) ein geringerer Loss einstellt.

Um das Training stabiler zu machen, haben wir als Monitoring-Metrik `val_accuracy` verwendet. Damit wird das Modell in der Regel in einer späteren Epoche ausgewählt, in der häufig auch der F1-Score höher und stabiler ist.

5.2.10 Tensorboard

Im Folgenden untersuchen wir das Verhalten aus 5.2.9 unter Verwendung des Tensorboards.

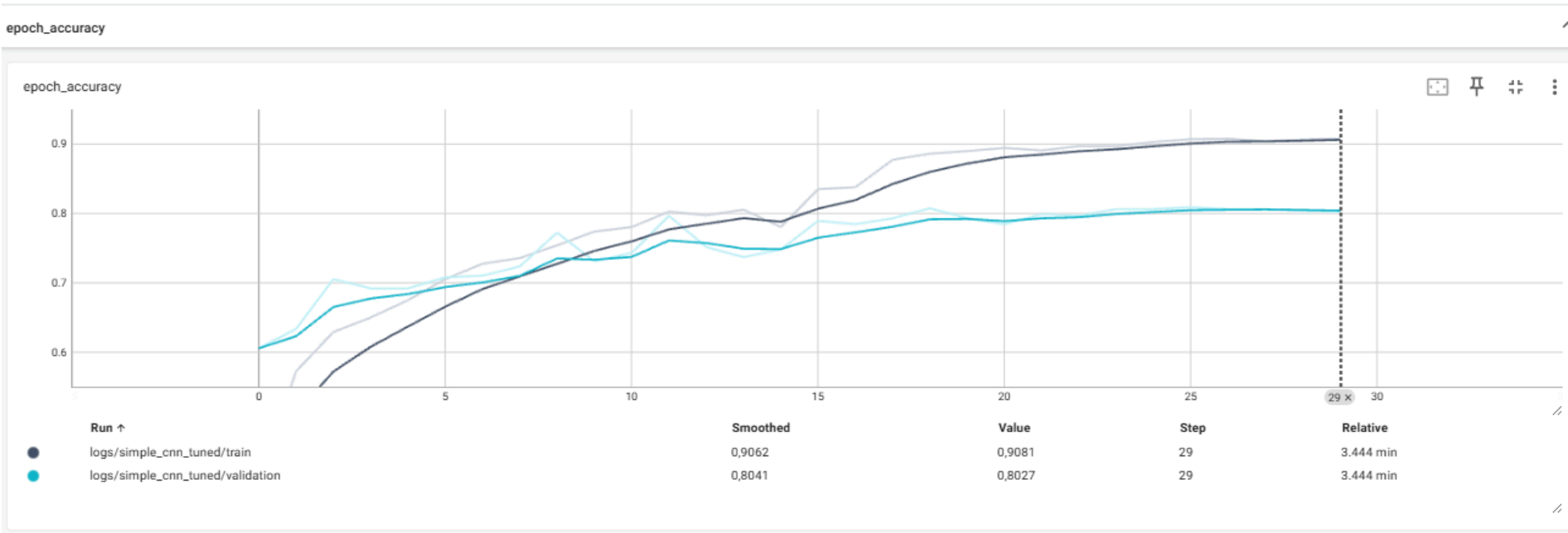
Das Tensorboard bietet uns eine Visualisierungen des Trainingsverhaltens. Um es zu nutzen haben wir bei der vorherigen Ausführung die Logs abgespeichert. Diese sind ebenfalls über Google Drive [hier](#) abrufbar.

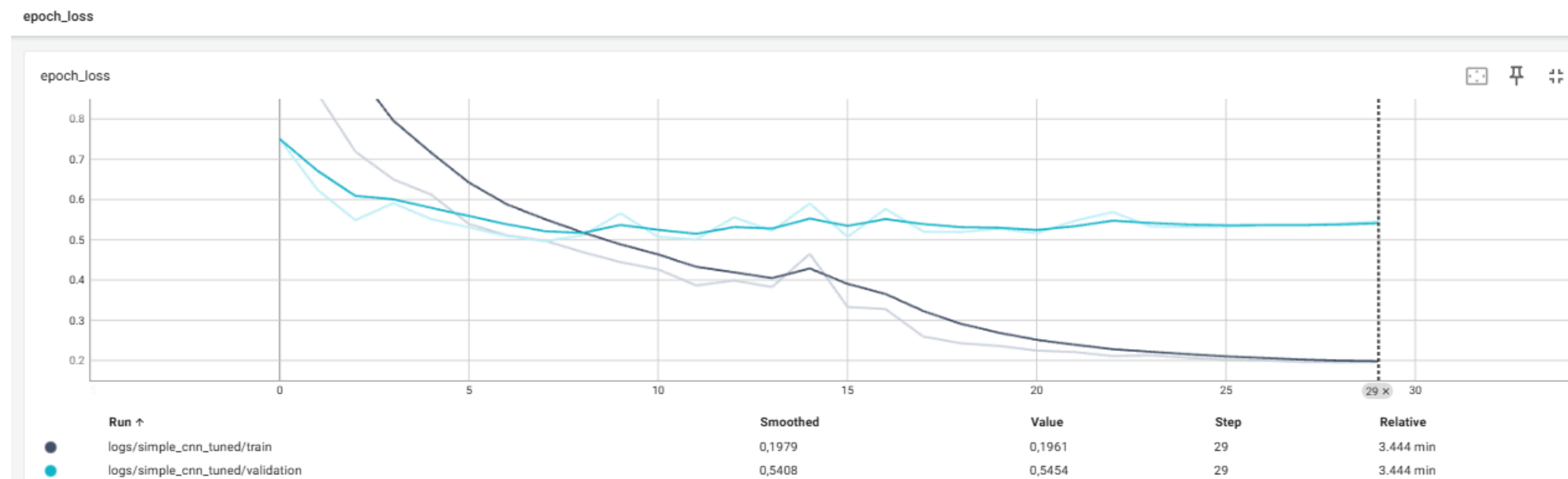
```
file_url: str = "https://drive.google.com/file/d/1VbHWIEJS8TGulII330qVxy9c2D9r_Kes/view?usp=sharing"
prefix: str = "tensorboard_logs"
load(file_url, prefix)
```

```
%load_ext tensorboard
```

```
%tensorboard --logdir data/tensorboard_logs
```

Für uns relevant ist jeweils die Accuracy und den Loss über die Epochen hinweg für Train und Validation.





Die Accuracy steigt sowohl auf Trainings- als auch auf Validierungsdaten an. Die Validierungskurve flacht früher ab. Gleichzeitig fällt der Trainings-Loss kontinuierlich, während der Validierungs-Loss nach einer anfänglichen Verbesserung stagniert.

Dieses Muster deutet darauf hin, dass in späteren Epochen nur noch wenig zusätzliche Generalisierung erreicht wird. Das Modell passt sich auf den Trainingsdaten weiter an, ohne dass sich die Qualität der Vorhersagewahrscheinlichkeiten auf den Validierungsdaten verbessert. Da der Loss (noch) nicht wieder zunimmt, gehen wir nicht von klassischen Overfitting aus.

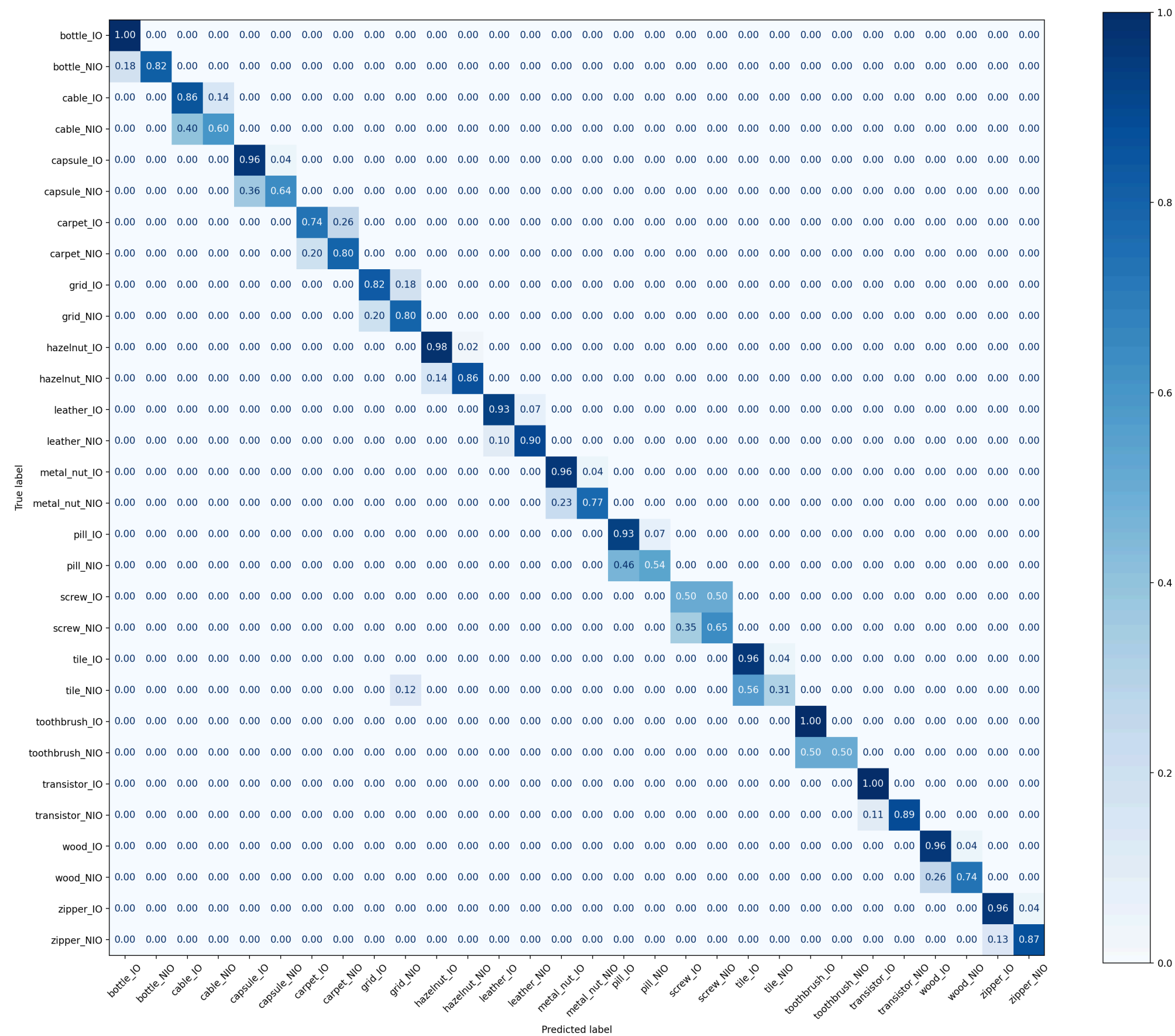
Eine plausible Erklärung ist, dass das Modell in vielen Fällen nahe an der Entscheidungsgrenze zwischen IO und NIO liegt. Es wird dadurch auf dem Validierungsset im Mittel nicht deutlich "sicherer" (stagnierender val_loss), kann aber dennoch die Trefferquote leicht erhöhen (steigende val_accuracy), indem einzelne Beispiele gerade noch auf die richtige Seite der Entscheidungsgrenze kippen. Gerade dieses "knappe Über die Grenze kommen" wirkt sich auf die Accuracy aus, ohne dass die Modellkonfidenz insgesamt entsprechend zunimmt.

Da wir konstant einen besseren F1-Score mit der `val_accuracy` erreichen, verwenden ab jetzt `val_accuracy` als Monitoring-Metrik in den Callback-Funktionen.

5.2.11 Confusion Matrix

```
model = import_model(
    "https://drive.google.com/file/d/11dIeq5jv0vJ0tj80eWbqwk0u1JKlM5AL/view?usp=sharing",
    "simple_cnn_tuned_bundle")
```

```
cm_counts = plot_confusionmatrix(
    model,
    "data/data_augmented_split/val",
    batch_size=32,
    seed=43,
)
```



Die Confusionmatrix zeigt recht deutlich, dass wenn eine Klasse falsch klassifiziert wird, sie innerhalb einer Kategorie falsch klassifiziert wird.

Interessant ist eine kleine Anomalie über die Kategorien hinaus: Tile NIO und Grid NIO werden manchmal vertauscht.

Evaluationswerte: (werden in Kapitel 5.2.14 besprochen)

```
df = eval(model, "data/data_augmented_split/test", batch_size=32, seed=43)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	38	0.552632	0.466667	0.506024	0.949320
1	hazelnut_IO	43	47	0.893617	0.976744	0.933333	0.992583
2	hazelnut_NIO	36	32	0.968750	0.861111	0.911765	0.992583
3	screw_IO	36	43	0.441860	0.527778	0.481013	0.949320
4	cable_NIO	31	25	0.880000	0.709677	0.785714	0.985167
5	carpet_IO	31	34	0.911765	1.000000	0.953846	0.996292
6	pill_IO	30	44	0.590909	0.866667	0.702703	0.972806
7	pill_NIO	30	16	0.750000	0.400000	0.521739	0.972806
8	grid_IO	29	35	0.685714	0.827586	0.750000	0.980222
9	metal_nut_NIO	28	23	0.913043	0.750000	0.823529	0.988875
10	cable_IO	28	34	0.735294	0.892857	0.806452	0.985167
11	capsule_NIO	28	25	0.920000	0.821429	0.867925	0.991347
12	transistor_IO	28	26	0.923077	0.857143	0.888889	0.992583
13	wood_NIO	27	23	0.956522	0.814815	0.880000	0.992583
14	tile_IO	27	36	0.666667	0.888889	0.761905	0.981459
15	leather_IO	27	31	0.806452	0.925926	0.862069	0.990111
16	zipper_IO	27	24	0.916667	0.814815	0.862745	0.991347
17	leather_NIO	27	23	0.913043	0.777778	0.840000	0.990111
18	transistor_NIO	27	29	0.862069	0.925926	0.892857	0.992583
19	wood_IO	26	30	0.833333	0.961538	0.892857	0.992583
20	grid_NIO	25	19	0.736842	0.560000	0.636364	0.980222
21	metal_nut_IO	24	29	0.758621	0.916667	0.830189	0.988875
22	carpet_NIO	24	21	1.000000	0.875000	0.933333	0.996292
23	capsule_IO	24	27	0.814815	0.916667	0.862745	0.991347
24	bottle_IO	23	25	0.880000	0.956522	0.916667	0.995056
25	bottle_NIO	22	20	0.950000	0.863636	0.904762	0.995056
26	tile_NIO	21	12	0.750000	0.428571	0.545455	0.981459
27	zipper_NIO	21	24	0.791667	0.904762	0.844444	0.991347
28	toothbrush_NIO	7	4	1.000000	0.571429	0.727273	0.996292
29	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996292
30	ALL	809	809	0.816779	0.802020	0.798338	0.796044

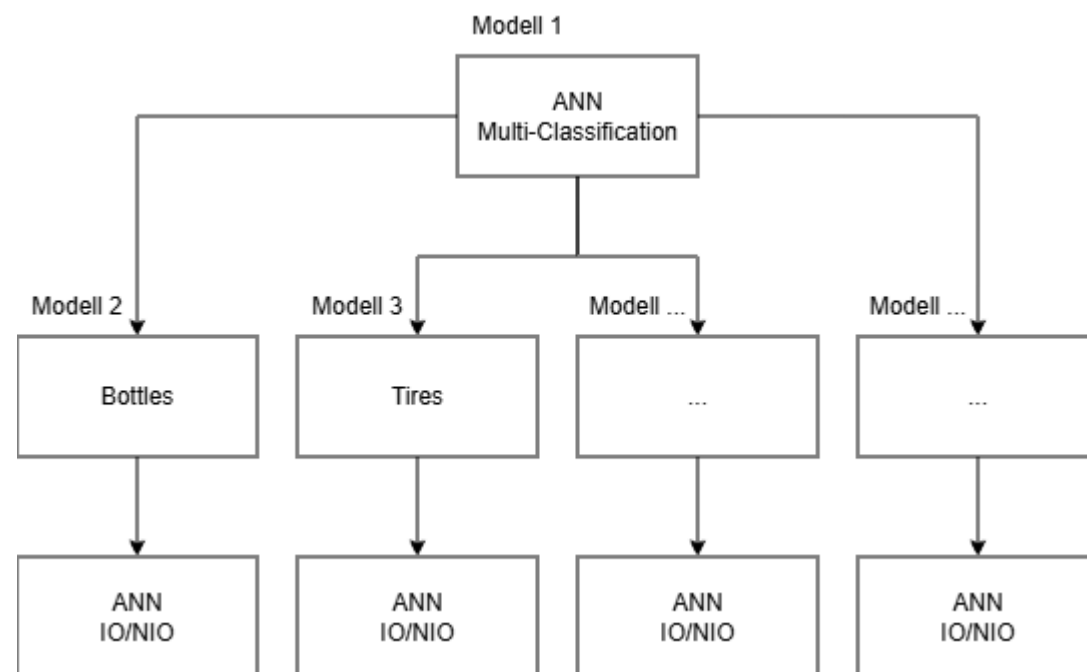
5.2.12 Variation A: Classifier-Ensemble

Die Idee dieser Variation des `simple_cnn` ist es, dem System die Information mitzuteilen, dass es 15 Kategorien gibt und diese sich dann erst in IO/NIO klassifizieren lassen.

Die vorhersage des Modells läuft dabei wie folgt ab:

Es wird zunächst nur die Kategorie durch ein Modell klassifiziert. Anschließend wird das IO/NIO Modell der vorhergesagten Kategorie ausgewählt und angewendet. Die beiden Ergebnisse (Kategorie + IO/NIO) können anschließend wieder zusammengeführt werden um das 30er Klassifikationsproblem zu lösen.

Der größte Nachteil des Modells ist es, dass ein IO/NIO Modell nicht mehr den Fehler des Kategoriemodells korrigieren kann. Würde der Kategorieklassifikator nur 80% der Kategorien richtig vorhersagen und jeder IO/NIO Klassifikator ebenfalls nur 80% richtig vorhersagen, so wären wir nur noch bei 64 % (0.8×0.8). Da wir jedoch einen "perfekten" Kategorieklassifikator aus Sektion 5.2.3 haben, ist dieser Nachteil nicht existent.



Als Kategorie-Klassifizierer importieren wir das in Kapitel 5.2.3 trainierte Modell wieder. Für jede Kategorie trainieren wir in einem Loop ein IO/NIO Klassifikationsmodell auf den augmentierten IO/NIO Daten der jeweiligen Kategorie. Für die IO/NIO Modelle verwenden wir die Architektur des `simple_cnn` wieder.

Im Anschluss bauen wir uns das Ensemble als kleines "Pseudomodell". Es enthält alle 1 + 15 Modelle. Es wendet den Kategorieklassifikator an, wählt die Kategorie mit der höchsten Confidence aus, wendet das IO/NIO Modell aus und wendet es an. Anschließend kombiniert er die beiden Outputs. Das Pseudomodell hat damit die gleiche Schnittstelle wie das einfache `simple_cnn`.

✓ 5.2.12.1 IO/NIO Modell

```

from tensorflow import keras
from keras import layers

def make_simple_cnn_io_nio(input_shape=(180, 180, 3), lr=1e-3, model_name="simple_cnn_io_nio"):
    inputs = keras.Input(shape=input_shape)
    x = layers.Rescaling(1.0/255)(inputs)
  
```



```

x = layers.Conv2D(16, 3, padding="same", activation="relu")(x)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
x = layers.MaxPooling2D()(x)

x = layers.Flatten()(x)
x = layers.Dense(64, activation="relu")(x)
x = layers.Dropout(0.3)(x)

outputs = layers.Dense(2, activation="softmax")(x)
model = keras.Model(inputs, outputs, name=model_name)
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
return model

```

✓ 5.2.12.2 IO/NIO Training

```

from tensorflow import keras

# We need the category sorting from here, as we trained the category model based on not augmented data
cat_ds = keras.utils.image_dataset_from_directory(
    "data/data_not_augmented_split_categories_only/val",
    image_size=(180,180),
    batch_size=32,
    shuffle=False,
)
CATEGORIES = cat_ds.class_names
print(CATEGORIES)

```

```

lr = 1e-3
epochs = 20
seed = 43

io_models = {} # Per category a model

for cat in CATEGORIES:
    class_names_2 = [f"{cat}_IO", f"{cat}_NIO"]

    name = f"simple_cnn_io_nio_{cat}"
    model_io = make_simple_cnn_io_nio(lr=lr, model_name=name)

    inp = TrainInput(
        data_dir="data/data_augmented_split",
        model=model_io,

```

```

        seed=seed,
        epochs=epochs,
        callback_metric="val_accuracy",
        tensorboard_logdir=f"logs/io_{cat}",
        class_names=class_names_2,
    )

    out = train(inp)
    io_models[cat] = out.model

    run = dict(
        name=out.model.name,
        model=out.model,
        model_name=out.model.name,
        lr=lr,
        seed=seed,
        epochs=epochs,
    )
    zip_path = export_model(run)
    print("Exported:", zip_path)

    df2 = eval(out.model, "data/data_augmented_split/val", class_names=class_names_2)
    print(cat, df2.tail(1)) # ALL-Line

```

► Training-Log

```

import pandas as pd

rows = []

for cat, model in io_models.items():
    df2 = eval(model, "data/data_augmented_split/val")
    all_row = df2.loc[df2["class"] == "ALL"].copy()
    all_row.insert(0, "category", cat)
    rows.append(all_row)

summary_df = pd.concat(rows, ignore_index=True).sort_values("f1", ascending=False).reset_index(drop=True)
summary_df

```

	category	class	true_count	pred_count	precision	recall	f1	accuracy
0	transistor	ALL	54	54	0.982143	0.981481	0.981475	0.981481
1	hazelnut	ALL	93	93	0.956250	0.958372	0.956865	0.956989
2	bottle	ALL	45	45	0.942308	0.931818	0.932802	0.933333
3	toothbrush	ALL	13	13	0.937500	0.916667	0.921212	0.923077
4	cable	ALL	53	53	0.906609	0.904286	0.905120	0.905660
5	leather	ALL	58	58	0.916667	0.892857	0.894545	0.896552
6	carpet	ALL	66	66	0.894444	0.892627	0.893327	0.893939
7	zipper	ALL	58	58	0.887019	0.884707	0.879274	0.879310
8	metal_nut	ALL	46	46	0.900000	0.863636	0.865497	0.869565
9	pill	ALL	57	57	0.867424	0.858374	0.858561	0.859649
10	tile	ALL	58	58	0.842105	0.812500	0.790865	0.793103
11	wood	ALL	54	54	0.837500	0.759259	0.744448	0.759259
12	grid	ALL	53	53	0.735000	0.735000	0.735000	0.735849
13	capsule	ALL	46	46	0.800000	0.636364	0.589286	0.652174
14	screw	ALL	67	67	0.268657	0.500000	0.349515	0.537313

Die meisten IO/NIO Klassifikatoren haben gute Ergebnisse. Negativ sticht vor allem die Schraube und die Pille heraus.

✓ 5.2.12.3 Ensemble

Das Kategorie Modell findet sich [hier](#).

Die NIO/IO Modelle finden sich [hier](#).

```
import numpy as np

class CatThenIOEnsemble:
    def __init__(self, category_model, io_nio_models, categories):
        self.category_model = category_model
        # io_nio_models: dict {"bottle": model, "capsule": model, ...}
        self.io_nio_models = io_nio_models

        # List of categories in same order as category_model output
        self.categories = categories

    def predict(self, images, verbose=0):
        # [image1, image2] => [ [0.5, 0.1, 0.1, ... ], [0.5, 0.1, 0.1, ...]]
        category_probs = self.category_model.predict(images, verbose=verbose)
        category_ids = np.argmax(category_probs, axis=1) # Input [Image1, Image2, Image3] => [0, 4, 2] (category ids)

        # [ bottle_io, botte_nio, dable_io, cable_nio, ... ... ]
        output = np.zeros((len(images), 30), dtype=np.float32)

        for i, category_id in enumerate(category_ids):
            # Get Category name => Get model => Predict
            category_name = self.categories[int(category_id)]
```

```
io_nio = self.io_nio_models[category_name].predict(images[i:i+1], verbose=verbose)[0] # [IO, NIO]

# e.g. cable_nio => Category id 1 => 2*1 = 2
# place 2 for IO, place 3 for NIO
start = 2 * int(category_id)
output[i, start] = io_nio[0] # IO
output[i, start + 1] = io_nio[1] # NIO

return output
```

```
CLASS_NAMES_30 = []
for cat in CATEGORIES:
    CLASS_NAMES_30 += [f"{cat}_IO", f"{cat}_NIO"]
```

```
model_category = import_model(
    "https://drive.google.com/file/d/1B94yhHx2SJVVcEqCEdUBrJwt0q4Mn0Rf/view?usp=sharing",
    "simple_cnn_category_only_bundle")
```

```
ensemble = CatThenIOEnsemble(model_category, io_models, CATEGORIES)

df_ens = eval(
    ensemble,
    "data/data_augmented_split/val",
    class_names=CLASS_NAMES_30
)
df_ens
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	48	0.979167	0.940000	0.959184	0.995128
1	hazelnut_IO	43	45	0.933333	0.976744	0.954545	0.995128
2	screw_IO	36	67	0.537313	1.000000	0.699029	0.962241
3	carpet_NIO	35	36	0.888889	0.914286	0.901408	0.991474
4	tile_NIO	32	20	1.000000	0.625000	0.769231	0.985384
5	zipper_NIO	31	26	0.961538	0.806452	0.877193	0.991474
6	screw_NIO	31	0	0.000000	0.000000	0.000000	0.962241
7	carpet_IO	31	30	0.900000	0.870968	0.885246	0.991474
8	leather_NIO	30	36	0.833333	1.000000	0.909091	0.992692
9	pill_IO	29	33	0.818182	0.931034	0.870968	0.990256
10	pill_NIO	28	24	0.916667	0.785714	0.846154	0.990256
11	leather_IO	28	22	1.000000	0.785714	0.880000	0.992692
12	grid_IO	28	28	0.750000	0.750000	0.750000	0.982948
13	cable_IO	28	29	0.896552	0.928571	0.912281	0.993910
14	transistor_IO	27	28	0.964286	1.000000	0.981818	0.998782
15	wood_NIO	27	14	1.000000	0.518519	0.682927	0.984166
16	zipper_IO	27	32	0.812500	0.962963	0.881356	0.991474
17	wood_IO	27	40	0.675000	1.000000	0.805970	0.984166
18	transistor_NIO	27	26	1.000000	0.962963	0.981132	0.998782
19	tile_IO	26	38	0.684211	1.000000	0.812500	0.985384
20	grid_NIO	25	25	0.720000	0.720000	0.720000	0.982948
21	cable_NIO	25	24	0.916667	0.880000	0.897959	0.993910
22	metal_nut_IO	24	30	0.800000	1.000000	0.888889	0.992692
23	capsule_IO	24	40	0.600000	1.000000	0.750000	0.980512
24	bottle_IO	23	26	0.884615	1.000000	0.938776	0.996346
25	bottle_NIO	22	19	1.000000	0.863636	0.926829	0.996346
26	capsule_NIO	22	6	1.000000	0.272727	0.428571	0.980512
27	metal_nut_NIO	22	16	1.000000	0.727273	0.842105	0.992692
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.998782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.844908	0.835197	0.819853	0.836784

Evaluationswerte: (werden in Kapitel 5.2.14 besprochen)

```
ensemble = CatThenIOEnsemble(model_category, io_models, CATEGORIES)

df_ens = eval(
    ensemble,
    "data/data_augmented_split/test",
    class_names=CLASS_NAMES_30
```

```
)
df_ens
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	0	0.000000	0.000000	0.000000	0.944376
1	hazelnut_IO	43	40	0.975000	0.906977	0.939759	0.993820
2	hazelnut_NIO	36	39	0.897436	0.972222	0.933333	0.993820
3	screw_IO	36	81	0.444444	1.000000	0.615385	0.944376
4	cable_NIO	31	23	0.913043	0.677419	0.777778	0.985167
5	carpet_IO	31	36	0.833333	0.967742	0.895522	0.991347
6	pill_IO	30	33	0.757576	0.833333	0.793651	0.983931
7	pill_NIO	30	27	0.814815	0.733333	0.771930	0.983931
8	grid_IO	29	33	0.727273	0.827586	0.774194	0.982695
9	metal_nut_NIO	28	21	0.952381	0.714286	0.816327	0.988875
10	cable_IO	28	36	0.722222	0.928571	0.812500	0.985167
11	capsule_NIO	28	3	1.000000	0.107143	0.193548	0.969098
12	transistor_IO	28	31	0.903226	1.000000	0.949153	0.996292
13	wood_NIO	27	16	0.875000	0.518519	0.651163	0.981459
14	tile_IO	27	41	0.634146	0.962963	0.764706	0.980222
15	leather_IO	27	22	0.863636	0.703704	0.775510	0.986403
16	zipper_IO	27	26	0.807692	0.777778	0.792453	0.986403
17	leather_NIO	27	32	0.750000	0.888889	0.813559	0.986403
18	transistor_NIO	27	24	1.000000	0.888889	0.941176	0.996292
19	wood_IO	26	37	0.648649	0.923077	0.761905	0.981459
20	grid_NIO	25	21	0.761905	0.640000	0.695652	0.982695
21	metal_nut_IO	24	31	0.741935	0.958333	0.836364	0.988875
22	carpet_NIO	24	19	0.947368	0.750000	0.837209	0.991347
23	capsule_IO	24	49	0.489796	1.000000	0.657534	0.969098
24	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998764
25	bottle_NIO	22	21	1.000000	0.954545	0.976744	0.998764
26	tile_NIO	21	7	0.857143	0.285714	0.428571	0.980222
27	zipper_NIO	21	22	0.727273	0.761905	0.744186	0.986403
28	toothbrush_NIO	7	4	1.000000	0.571429	0.727273	0.996292
29	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996292
30	ALL	809	809	0.790121	0.775145	0.749311	0.765142

5.2.13 Variation B: 2 Head-Classifier

Wie beim Classifier-Ensemble (Variante A) ist die Grundidee, dem `simple_cnn` explizit die Struktur des Problems mitzugeben: Die 30 Klassen lassen sich als Kombination aus 15 Kategorien und dem Zustand IO/NIO darstellen. Statt jedoch zuerst eine Kategorie zu

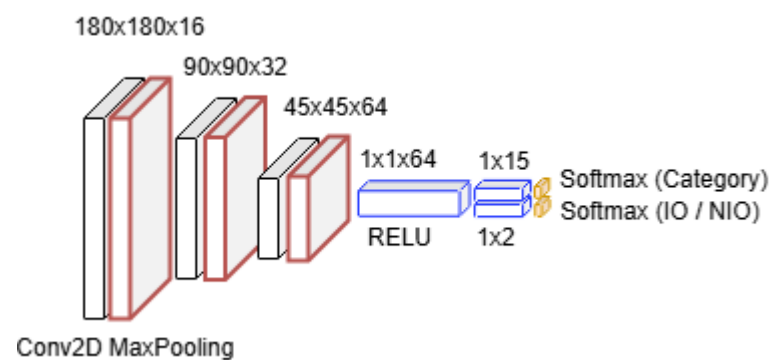
bestimmen und danach ein separates Modell pro Kategorie anzuwenden, lösen wir beide Teilaufgaben gleichzeitig innerhalb eines einzigen Netzes.

Der Ansatz folgt dem Prinzip des Multi-Task Learning. Wir verwenden einen gemeinsamen Backbone, während zwei getrennte Heads klassifizieren. Das Training erfolgt gegen 2 Fehlerfunktionen. Eine für die Kategorie-Klassifikation und eine für die IO/NIO-Klassifikation.

Nach dem trainieren des Modells, werden wir auch hier wieder ein kleines "Pseudomodell" erstellen. Die Aufgabe des Pseudomodells ist es hier die zwei vereinzelt Ausgaben der Heads zu dem regulären 30er Outputformat zu transformieren.

✓ 5.2.13.1 Modell

Der Backbone des Modells ist identisch zu unserem ursprünglichen `simple_cnn` Modell aufgebaut. Der Unterschied sind nun die zwei Dense Layer, welche unsere Outputs bereitstellen.



```
from tensorflow import keras
from keras import layers

def make_simple_cnn_two_heads(input_shape=(180, 180, 3), num_categories=15):
    inputs = keras.Input(shape=input_shape)
    x = layers.Rescaling(1.0/255)(inputs)

    # Backbone
    x = layers.Conv2D(filters=16, kernel_size=3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x)

    x = layers.Conv2D(filters=32, kernel_size=3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x)

    x = layers.Conv2D(filters=64, kernel_size=3, padding="same", activation="relu")(x)
    x = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(x)

    x = layers.Flatten()(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.3)(x)

    # Heads
    category_out = layers.Dense(num_categories, activation="softmax", name="category")(x)
    quality_out = layers.Dense(2, activation="softmax", name="io")(x)
```

```
model = keras.Model(inputs, [category_out, quality_out], name="simple_cnn_two_heads")
return model
```

✓ 5.2.13.2 Train-Pipeline

Für das Training müssen wir die Methode aus 5.1.1 anpassen um aus der 30er Klassifikation eine (15, 2) Klassifikation zu machen. Damit wir die Implementierung dort nicht komplexer machen, haben wir diese folgend neu definiert für die Two-Head Classification angepasst.

```
from dataclasses import dataclass
from typing import Dict, Optional
import numpy as np
import tensorflow as tf
from tensorflow import keras

@dataclass
class TrainInput:
    data_dir: str
    model: tf.keras.Model
    batch_size: int = 32
    seed: int = 42
    epochs: int = 50
    class_weight: Optional[Dict[int, float]] = None
    tensorboard_logdir: Optional[str] = None
    callback_metric: str = "val_loss"

@dataclass
class TrainOutput:
    model: tf.keras.Model
    hist: keras.callbacks.History

def split_label(image, y30):
    y_category = y30 // 2    # 0..14 => Immer eines Skippen => bottle, capsule
    y_io       = y30 % 2    # 0..1  => Jedes 2. IO, NIO, IO, NIO
    return image, {"category": y_category, "io": y_io}

def train_2heads(inp: TrainInput) -> TrainOutput:
    tf.keras.utils.set_random_seed(inp.seed)

    #Required to set, as we need the ordering for split_label
    CLASS_NAMES = [
        "bottle_IO",      "bottle_NIO",
        "capsule_IO",     "capsule_NIO",
        "grid_IO",        "grid_NIO",
        "leather_IO",     "leather_NIO",
        "pill_IO",        "pill_NIO",
        "tile_IO",        "tile_NIO",
        "transistor_IO",  "transistor_NIO",
        "zipper_IO",      "zipper_NIO",
        "cable_IO",       "cable_NIO",
        "carpet_IO",      "carpet_NIO",
        "hazelnut_IO",    "hazelnut_NIO",
        "metal_nut_IO",   "metal_nut_NIO",
```



```

"screw_IO",      "screw_NIO",
"toothbrush_IO", "toothbrush_NIO",
"wood_IO",       "wood_NIO",
]

training_train_tensor = tf.keras.utils.image_dataset_from_directory(
    inp.data_dir+"/train",
    class_names=CLASS_NAMES,
    seed=inp.seed,
    image_size=(180, 180),
    batch_size=inp.batch_size)

training_eval_tensor = tf.keras.utils.image_dataset_from_directory(
    inp.data_dir+"/val",
    class_names=CLASS_NAMES,
    seed=inp.seed,
    image_size=(180, 180),
    batch_size=inp.batch_size)

training_train_tensor = training_train_tensor.map(split_label, num_parallel_calls=tf.data.AUTOTUNE)
training_eval_tensor  = training_eval_tensor.map(split_label, num_parallel_calls=tf.data.AUTOTUNE)

callbacks = [
    # Stops earlier than specified epochs, when no longer a
    # significant improve on 'val_loss' occurs for 10 epochs
    keras.callbacks.EarlyStopping(
        monitor=inp.callback_metric,
        patience=10,
        restore_best_weights=True,
        verbose=1
    ),
    # Saves the best model based on val_loss
    keras.callbacks.ModelCheckpoint(
        f"{inp.model.name}.keras",
        monitor=inp.callback_metric,
        save_best_only=True
    ),
    # Reduces learning rate when a val_loss has stopped improving.
    keras.callbacks.ReduceLROnPlateau(
        monitor=inp.callback_metric,
        factor=0.1,
        patience=5,
        verbose=1
    )
]

if inp.tensorboard_logdir:
    callbacks.append(
        keras.callbacks.TensorBoard(
            log_dir=inp.tensorboard_logdir,
            histogram_freq=0,
            write_graph=True)
    )

```

```

hist = inp.model.fit(
    training_train_tensor,
    validation_data=training_eval_tensor,
    epochs=inp.epochs,
    callbacks=callbacks,
    verbose=1,
    class_weight=inp.class_weight, # When none => ignored
)

return TrainOutput(
    model=inp.model,
    hist=hist)

```

5.2.13.3 Training

```

epochs = 30
seed = 43
lr = 1e-3

model = make_simple_cnn_two_heads()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss={"category": "sparse_categorical_crossentropy", "io": "sparse_categorical_crossentropy"},
    metrics={"category": ["accuracy"], "io": ["accuracy"]},
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir="logs/simple_cnn_two_heads"
)

out = train_2heads(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs,
)

zip_path = export_model(run)
print("Exported:", zip_path)

```

► Training-Log

```

model = import_model(
    "https://drive.google.com/file/d/17QiKR-6pltuogsZplfrfhKfWxvyqc6cp/view?usp=sharing",

```

```
"simple_cnn_two_heads_bundle")
```

5.2.13.4 2 Head-Classifer Wrapper

```
import numpy as np

categories = [
    "bottle",
    "capsule",
    "grid",
    "leather",
    "pill",
    "tile",
    "transistor",
    "zipper",
    "cable",
    "carpet",
    "hazelnut",
    "metal_nut",
    "screw",
    "toothbrush",
    "wood",
]

CLASS_NAMES_30 = [
    "bottle_IO",      "bottle_NIO",
    "capsule_IO",     "capsule_NIO",
    "grid_IO",        "grid_NIO",
    "leather_IO",     "leather_NIO",
    "pill_IO",        "pill_NIO",
    "tile_IO",        "tile_NIO",
    "transistor_IO",  "transistor_NIO",
    "zipper_IO",      "zipper_NIO",
    "cable_IO",       "cable_NIO",
    "carpet_IO",      "carpet_NIO",
    "hazelnut_IO",    "hazelnut_NIO",
    "metal_nut_IO",   "metal_nut_NIO",
    "screw_IO",       "screw_NIO",
    "toothbrush_IO",  "toothbrush_NIO",
    "wood_IO",        "wood_NIO",
]

class TwoHeadTo30:
    def __init__(self, two_head_model, categories):
        self.m = two_head_model
        self.categories = categories

    def predict(self, images, verbose=0):
        probability_category, probability_io_nio = self.m.predict(images, verbose=verbose) # (B,15), (B,2)

        category_id = np.argmax(probability_category, axis=1) # (B,)
        probability_30_classes = np.zeros((len(images), 30), dtype=np.float32)
```

```
for image_index, predicted_category_id in enumerate(category_id):
    io_index = 2 * int(predicted_category_id)
    nio_index = io_index + 1

    probability_30_classes[image_index, io_index] = probability_io_nio[image_index, 0]
    probability_30_classes[image_index, nio_index] = probability_io_nio[image_index, 1]

return probability_30_classes
```

```
wrapped = TwoHeadTo30(model, categories)
```

```
df = eval(wrapped, "data/data_augmented_split/val", batch_size=32, seed=43, class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	47	0.978723	0.920000	0.948454	0.993910
1	hazelnut_IO	43	46	0.913043	0.976744	0.943820	0.993910
2	screw_IO	36	43	0.627907	0.750000	0.683544	0.969549
3	carpet_NIO	35	24	0.916667	0.628571	0.745763	0.981730
4	tile_NIO	32	1	0.000000	0.000000	0.000000	0.959805
5	zipper_NIO	31	26	0.923077	0.774194	0.842105	0.989038
6	screw_NIO	31	24	0.625000	0.483871	0.545455	0.969549
7	carpet_IO	31	42	0.690476	0.935484	0.794521	0.981730
8	leather_NIO	30	29	0.827586	0.800000	0.813559	0.986602
9	pill_IO	29	46	0.630435	1.000000	0.773333	0.979294
10	cable_IO	28	32	0.781250	0.892857	0.833333	0.987820
11	grid_IO	28	27	0.777778	0.750000	0.763636	0.984166
12	leather_IO	28	29	0.793103	0.821429	0.807018	0.986602
13	pill_NIO	28	11	1.000000	0.392857	0.564103	0.979294
14	wood_IO	27	35	0.771429	1.000000	0.870968	0.990256
15	wood_NIO	27	19	1.000000	0.703704	0.826087	0.990256
16	zipper_IO	27	32	0.781250	0.925926	0.847458	0.989038
17	transistor_NIO	27	25	1.000000	0.925926	0.961538	0.997564
18	transistor_IO	27	29	0.931034	1.000000	0.964286	0.997564
19	tile_IO	26	53	0.471698	0.961538	0.632911	0.964677
20	cable_NIO	25	21	0.857143	0.720000	0.782609	0.987820
21	grid_NIO	25	30	0.633333	0.760000	0.690909	0.979294
22	metal_nut_IO	24	28	0.821429	0.958333	0.884615	0.992692
23	capsule_IO	24	22	0.818182	0.750000	0.782609	0.987820
24	bottle_IO	23	25	0.920000	1.000000	0.958333	0.997564
25	bottle_NIO	22	20	1.000000	0.909091	0.952381	0.997564
26	metal_nut_NIO	22	18	0.944444	0.772727	0.850000	0.992692
27	capsule_NIO	22	24	0.750000	0.818182	0.782609	0.987820
28	toothbrush_IO	7	8	0.750000	0.857143	0.800000	0.996346
29	toothbrush_NIO	6	5	0.800000	0.666667	0.727273	0.996346
30	ALL	821	821	0.791166	0.795175	0.779108	0.794153

Evaluationswerte: (werden in Kapitel 5.2.14 besprochen)

```
df = eval(wrapped, "data/data_augmented_split/test", batch_size=32, seed=43, class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	48	0.645833	0.688889	0.666667	0.961681
1	hazelnut_IO	43	47	0.893617	0.976744	0.933333	0.992583
2	screw_IO	36	33	0.575758	0.527778	0.550725	0.961681
3	hazelnut_NIO	36	32	0.968750	0.861111	0.911765	0.992583
4	carpet_IO	31	42	0.738095	1.000000	0.849315	0.986403
5	cable_NIO	31	25	0.800000	0.645161	0.714286	0.980222
6	pill_IO	30	45	0.644444	0.966667	0.773333	0.978986
7	pill_NIO	30	15	0.933333	0.466667	0.622222	0.978986
8	grid_IO	29	34	0.647059	0.758621	0.698413	0.976514
9	transistor_IO	28	30	0.900000	0.964286	0.931034	0.995056
10	cable_IO	28	34	0.676471	0.821429	0.741935	0.980222
11	metal_nut_NIO	28	22	0.954545	0.750000	0.840000	0.990111
12	capsule_NIO	28	27	0.703704	0.678571	0.690909	0.978986
13	transistor_NIO	27	25	0.960000	0.888889	0.923077	0.995056
14	zipper_IO	27	26	0.884615	0.851852	0.867925	0.991347
15	tile_IO	27	38	0.605263	0.851852	0.707692	0.976514
16	leather_NIO	27	34	0.794118	1.000000	0.885246	0.991347
17	leather_IO	27	20	1.000000	0.740741	0.851064	0.991347
18	wood_NIO	27	16	0.875000	0.518519	0.651163	0.981459
19	wood_IO	26	37	0.648649	0.923077	0.761905	0.981459
20	grid_NIO	25	20	0.650000	0.520000	0.577778	0.976514
21	carpet_NIO	24	13	1.000000	0.541667	0.702703	0.986403
22	capsule_IO	24	25	0.640000	0.666667	0.653061	0.978986
23	metal_nut_IO	24	30	0.766667	0.958333	0.851852	0.990111
24	bottle_IO	23	25	0.880000	0.956522	0.916667	0.995056
25	bottle_NIO	22	20	0.950000	0.863636	0.904762	0.995056
26	tile_NIO	21	10	0.600000	0.285714	0.387097	0.976514
27	zipper_NIO	21	22	0.818182	0.857143	0.837209	0.991347
28	toothbrush_IO	7	9	0.777778	1.000000	0.875000	0.997528
29	toothbrush_NIO	7	5	1.000000	0.714286	0.833333	0.997528
30	ALL	809	809	0.797729	0.774827	0.770382	0.773795

5.2.14 Evaluation

In diesem Kapitel machen wir einen kleinen Vergleich des getunten Modells mit den 2 alternativen Varianten. Unten sind die Evaluationstabellen der Modelle gegen den Test-Datensatz abgebildet.

Getuntes Modell Evaluationswerte

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	38	0.552632	0.466667	0.506024	0.949320
1	hazelnut_IO	43	47	0.893617	0.976744	0.933333	0.992583
2	hazelnut_NIO	36	32	0.968750	0.861111	0.911765	0.992583
3	screw_IO	36	43	0.441860	0.527778	0.481013	0.949320
4	cable_NIO	31	25	0.880000	0.709677	0.785714	0.985167
5	carpet_IO	31	34	0.911765	1.000000	0.953846	0.996292
6	pill_IO	30	44	0.590909	0.866667	0.702703	0.972806
7	pill_NIO	30	16	0.750000	0.400000	0.521739	0.972806
8	grid_IO	29	35	0.685714	0.827586	0.750000	0.980222
9	metal_nut_NIO	28	23	0.913043	0.750000	0.823529	0.988875
10	cable_IO	28	34	0.735294	0.892857	0.806452	0.985167
11	capsule_NIO	28	25	0.920000	0.821429	0.867925	0.991347
12	transistor_IO	28	26	0.923077	0.857143	0.888889	0.992583
13	wood_NIO	27	23	0.956522	0.814815	0.880000	0.992583
14	tile_IO	27	36	0.666667	0.888889	0.761905	0.981459
15	leather_IO	27	31	0.806452	0.925926	0.862069	0.990111
16	zipper_IO	27	24	0.916667	0.814815	0.862745	0.991347
17	leather_NIO	27	23	0.913043	0.777778	0.840000	0.990111
18	transistor_NIO	27	29	0.862069	0.925926	0.892857	0.992583
19	wood_IO	26	30	0.833333	0.961538	0.892857	0.992583
20	grid_NIO	25	19	0.736842	0.560000	0.636364	0.980222
21	metal_nut_IO	24	29	0.758621	0.916667	0.830189	0.988875
22	carpet_NIO	24	21	1.000000	0.875000	0.933333	0.996292
23	capsule_IO	24	27	0.814815	0.916667	0.862745	0.991347
24	bottle_IO	23	25	0.880000	0.956522	0.916667	0.995056
25	bottle_NIO	22	20	0.950000	0.863636	0.904762	0.995056
26	tile_NIO	21	12	0.750000	0.428571	0.545455	0.981459
27	zipper_NIO	21	24	0.791667	0.904762	0.844444	0.991347
28	toothbrush_NIO	7	4	1.000000	0.571429	0.727273	0.996292
29	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996292
30	ALL	809	809	0.816779	0.802020	0.798338	0.796044

Ensemble Evaluationswerte

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	0	0.000000	0.000000	0.000000	0.944376
1	hazelnut_IO	43	40	0.975000	0.906977	0.939759	0.993820
2	hazelnut_NIO	36	39	0.897436	0.972222	0.933333	0.993820
3	screw_IO	36	81	0.444444	1.000000	0.615385	0.944376
4	cable_NIO	31	23	0.913043	0.677419	0.777778	0.985167
5	carpet_IO	31	36	0.833333	0.967742	0.895522	0.991347
6	pill_IO	30	33	0.757576	0.833333	0.793651	0.983931
7	pill_NIO	30	27	0.814815	0.733333	0.771930	0.983931
8	grid_IO	29	33	0.727273	0.827586	0.774194	0.982695
9	metal_nut_NIO	28	21	0.952381	0.714286	0.816327	0.988875
10	cable_IO	28	36	0.722222	0.928571	0.812500	0.985167
11	capsule_NIO	28	3	1.000000	0.107143	0.193548	0.969098
12	transistor_IO	28	31	0.903226	1.000000	0.949153	0.996292
13	wood_NIO	27	16	0.875000	0.518519	0.651163	0.981459
14	tile_IO	27	41	0.634146	0.962963	0.764706	0.980222
15	leather_IO	27	22	0.863636	0.703704	0.775510	0.986403
16	zipper_IO	27	26	0.807692	0.777778	0.792453	0.986403
17	leather_NIO	27	32	0.750000	0.888889	0.813559	0.986403
18	transistor_NIO	27	24	1.000000	0.888889	0.941176	0.996292
19	wood_IO	26	37	0.648649	0.923077	0.761905	0.981459
20	grid_NIO	25	21	0.761905	0.640000	0.695652	0.982695
21	metal_nut_IO	24	31	0.741935	0.958333	0.836364	0.988875
22	carpet_NIO	24	19	0.947368	0.750000	0.837209	0.991347
23	capsule_IO	24	49	0.489796	1.000000	0.657534	0.969098
24	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998764
25	bottle_NIO	22	21	1.000000	0.954545	0.976744	0.998764
26	tile_NIO	21	7	0.857143	0.285714	0.428571	0.980222
27	zipper_NIO	21	22	0.727273	0.761905	0.744186	0.986403
28	toothbrush_NIO	7	4	1.000000	0.571429	0.727273	0.996292
29	toothbrush_IO	7	10	0.700000	1.000000	0.823529	0.996292
30	ALL	809	809	0.790121	0.775145	0.749311	0.765142

2 Head Classifier Evaluationswerte

	class	true_count	pred_count	precision	recall	f1	accuracy
0	screw_NIO	45	48	0.645833	0.688889	0.666667	0.961681
1	hazelnut_IO	43	47	0.893617	0.976744	0.933333	0.992583
2	screw_IO	36	33	0.575758	0.527778	0.550725	0.961681
3	hazelnut_NIO	36	32	0.968750	0.861111	0.911765	0.992583
4	carpet_IO	31	42	0.738095	1.000000	0.849315	0.986403
5	cable_NIO	31	25	0.800000	0.645161	0.714286	0.980222
6	pill_IO	30	45	0.644444	0.966667	0.773333	0.978986
7	pill_NIO	30	15	0.933333	0.466667	0.622222	0.978986
8	grid_IO	29	34	0.647059	0.758621	0.698413	0.976514
9	transistor_IO	28	30	0.900000	0.964286	0.931034	0.995056
10	cable_IO	28	34	0.676471	0.821429	0.741935	0.980222
11	metal_nut_NIO	28	22	0.954545	0.750000	0.840000	0.990111
12	capsule_NIO	28	27	0.703704	0.678571	0.690909	0.978986
13	transistor_NIO	27	25	0.960000	0.888889	0.923077	0.995056
14	zipper_IO	27	26	0.884615	0.851852	0.867925	0.991347
15	tile_IO	27	38	0.605263	0.851852	0.707692	0.976514
16	leather_NIO	27	34	0.794118	1.000000	0.885246	0.991347
17	leather_IO	27	20	1.000000	0.740741	0.851064	0.991347
18	wood_NIO	27	16	0.875000	0.518519	0.651163	0.981459
19	wood_IO	26	37	0.648649	0.923077	0.761905	0.981459
20	grid_NIO	25	20	0.650000	0.520000	0.577778	0.976514
21	carpet_NIO	24	13	1.000000	0.541667	0.702703	0.986403
22	capsule_IO	24	25	0.640000	0.666667	0.653061	0.978986
23	metal_nut_IO	24	30	0.766667	0.958333	0.851852	0.990111
24	bottle_IO	23	25	0.880000	0.956522	0.916667	0.995056
25	bottle_NIO	22	20	0.950000	0.863636	0.904762	0.995056
26	tile_NIO	21	10	0.600000	0.285714	0.387097	0.976514
27	zipper_NIO	21	22	0.818182	0.857143	0.837209	0.991347
28	toothbrush_IO	7	9	0.777778	1.000000	0.875000	0.997528
29	toothbrush_NIO	7	5	1.000000	0.714286	0.833333	0.997528
30	ALL	809	809	0.797729	0.774827	0.770382	0.773795

Insgesamt haben alle 3 Modelle sinnvolle Ergebnisse geliefert.

Der F1-Score über Evaluationsdaten ist beim getunten Modell und beim 2 Head-Classifer nahezu identisch mit dem F1-Score der Validierungsdaten. Nur beim Ensemble gibt es eine deutlichere Abweichung im F1-Score. Dieser ist bei den Evaluationswerten um 0.07 Punkten niedriger, was auf leichtes Overfitting hinweist.

Fast alle Modelle können die verschiedenen Kategorien sauber unterteilen. Eine Ausnahme bilden folgende Klassen:

- capsule und screw bei dem Ensemble

- tile bei dem 2 Head Classifier
- screw bei dem getunten Modell

Diese Klassen wurden mit einem F1-Score von < 0.5 vorhergesagt. Die Performance der Modelle ist vernachlässigbar ähnlich zueinander, wobei anzumerken ist, dass bei der Ensemble Variation und bei der 2 Head Classifier Variation noch kein Tuning erfolgt ist.

✓ 5.3 Experiments - Transfer learning on pretrained Models

Zusammenfassung des Lernpfads:

Insgesamt zeigt sich, wie vermutet, dass Transfer Learning mit vortrainierten Modellen deutlich bessere Ergebnisse liefern als die selbst von Scratch trainierten Modelle.

Wir haben mehrere Backbones mit mehreren Heads kombiniert. Heads schienen für unseren Use-Case eine größere Rolle zu spielen. Die beste Kombination war der MobileNetV3Small Backbone mit dem "Direct" Head, welcher aus nur einem Output-Layer besteht.

Durch Fine-Tuning des Modells konnten wir die Performance noch etwas erhöhen.

Unsere Performancemetrik, der F1-Score, ist auf dem Evaluationsdaten ähnlich gut, wie auf dem Validierungsdaten.

Die Konfusionsmatrix zeigt, dass das Modell die IO/NIOs der Kategorien Pill, Grid und Transistor noch etwas durcheinander bringt.

Referenzen für diesen Lernpfad:

https://www.tensorflow.org/api_docs/python/tf/keras/applications

https://keras.io/guides/transfer_learning/

✓ 5.3.1 Pretrained Mobile Net V3 Small

✓ 5.3.1.1 Direct classification

```
import keras
from keras import layers

def make_mobilenetv3_small_050(num_classes=30, image_size=(180, 180), dropout: float = 0.3) -> keras.Model:
    inputs = keras.Input(shape=image_size + (3,))

    x = keras.applications.mobilenet_v3.preprocess_input(inputs)

    base = keras.applications.MobileNetV3Small(
        input_shape=image_size + (3,),
        include_top=False,
        weights="imagenet",
        pooling="avg",      # Makes GlobalAveragePooling intern e.g. height and width dimensions are gone
    )
    base.trainable = False
```

```
x = base(x, training=False)
x = layers.Dropout(dropout)(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)

return keras.Model(inputs, outputs, name="mobilenetv3_small_050_simple")
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_mobilenetv3_small_050()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir="logs/mobilenetv3_small_050",
    callback_metric="val_accuracy"
)

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs,
)

zip_path = export_model(run)
print("Exported:", zip_path)
```

```
model = import_model(
    "https://drive.google.com/file/d/1SRhdjAXfVGbhHg5RuAfUcLy9pG0r5uw0/view?usp=sharing",
    "mobilenetv3_small_050_simple_bundle")
```

```
df = eval(model, "data/data_augmented_split/val", batch_size=32, seed=43)
df
```

```
df = eval(model, "data/data_augmented_split/test", batch_size=32, seed=43)
df
```

5.3.1.2 One Hidden Layer

```
import keras
from keras import layers

def make_mobilenetv3_small_050(num_classes=30, image_size=(180, 180), dropout: float = 0.3) -> keras.Model:
    inputs = keras.Input(shape=image_size + (3,))

    x = keras.applications.mobilenet_v3.preprocess_input(inputs)

    base = keras.applications.MobileNetV3Small(
        input_shape=image_size + (3,),
        include_top=False,
        weights="imagenet",
        pooling="avg",
    )
    base.trainable = False

    x = base(x, training=False)
    x = layers.Dropout(dropout)(x)
    x = layers.Dense(64, activation="relu")(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    return keras.Model(inputs, outputs, name="mobilenetv3_small_050_one_dense")
```

```
import keras
from keras import layers

lr = 1e-3
epochs = 30
seed = 43

model = make_mobilenetv3_small_050()
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir="logs/mobilenetv3_small_050",
    callback_metric="val_accuracy"
)
```

```

out = train(inp)

run = dict(
    name=out.model.name,
    model=out.model,
    model_name=out.model.name,
    lr=lr,
    seed=seed,
    epochs=epochs,

)
zip_path = export_model(run)
print("Exported:", zip_path)

```

```

df = eval(model, "data/data_augmented_split/val", batch_size=32, seed=43)
df

```

```

df = eval(model, "data/data_augmented_split/test", batch_size=32, seed=43)
df

```

✓ 5.3.2 Combining pretrained models and heads

✓ 5.3.2.1 Head Collection

```

from tensorflow import keras
from keras import layers

def head_direct(x, num_classes: int = 30, dropout: float = 0.3):
    x = layers.Dropout(dropout)(x)
    return layers.Dense(num_classes, activation="softmax", name="category")(x)

```

```

from tensorflow import keras
from keras import layers

def head_dense(x, num_classes: int = 30, dropout: float = 0.3):
    x = layers.Dropout(dropout)(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dense(64, activation="relu")(x)
    return layers.Dense(num_classes, activation="softmax", name="category")(x)

```

```

from tensorflow import keras
from keras import layers

def head_two_tasks(x,
                    num_categories: int = 30,
                    num_io: int = 2,
                    dropout: float = 0.3,
                    l2: float = 0.0):
    x = layers.Dropout(dropout)(x)

```

```
x = layers.Dense(64, activation="relu")(x)

category_out = layers.Dense(num_categories, activation="softmax", name="category")(x)
io_out = layers.Dense(num_io, activation="softmax", name="io")(x)
return [category_out, io_out]
```

```
def build_head(x, head_type: str):
    if head_type == "direct":
        return head_direct(x)

    if head_type == "dense":
        return head_dense(x)

    if head_type == "two_tasks":
        return head_two_tasks(x)
```

✓ 5.3.2.2 Pretrained Modell Collection

```
from tensorflow import keras

def make_premodel(backbone: str,
                  head_type="direct",
                  image_size=(180, 180)) -> keras.Model:

    inputs = keras.Input(shape=image_size + (3,), name="image")

    if backbone == "mobilenetv3small":
        x = keras.applications.mobilenet_v3.preprocess_input(inputs)
        base = keras.applications.MobileNetV3Small(
            include_top=False,
            weights="imagenet",
            input_shape=image_size + (3,),
            pooling="avg"
        )

    elif backbone == "xception":
        x = keras.applications.xception.preprocess_input(inputs)
        base = keras.applications.Xception(
            include_top=False,
            weights="imagenet",
            input_shape=image_size + (3,),
            pooling="avg"
        )

    elif backbone == "vgg16":
        x = keras.applications.vgg16.preprocess_input(inputs)
        base = keras.applications.VGG16(
            include_top=False,
            weights="imagenet",
            input_shape=image_size + (3,),
            pooling="avg"
```



```

    )

    base.trainable = False
    x = base(x, training=False)

    outputs = build_head(
        x,
        head_type=head_type
    )

    name = f"{backbone}_{head_type}"
    return keras.Model(inputs, outputs, name=name)

```

✓ 5.3.2.3 Training

✓ MobileNetV3Small-Backbone x Direct-Head

```

epochs = 30
seed = 43
lr = 1e-3

model = make_premodel("mobilenetv3small", "direct")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy"
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)

```

► Training-Log

```

model = import_model(
    "https://drive.google.com/file/d/18FVBX1Bm6kIrs8KDFqa-3ldUvGUtNwr_/view?usp=sharing",
    "mobilenetv3small_direct_bundle")

```

```
df = eval(model, "data/data_augmented_split/val")
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	46	1.000000	0.920000	0.958333	0.995128
1	hazelnut_IO	43	47	0.914894	1.000000	0.955556	0.995128
2	screw_IO	36	44	0.795455	0.972222	0.875000	0.987820
3	carpet_NIO	35	26	0.961538	0.714286	0.819672	0.986602
4	tile_NIO	32	29	1.000000	0.906250	0.950820	0.996346
5	carpet_IO	31	40	0.750000	0.967742	0.845070	0.986602
6	zipper_NIO	31	29	1.000000	0.935484	0.966667	0.997564
7	screw_NIO	31	23	0.956522	0.709677	0.814815	0.987820
8	leather_NIO	30	31	0.967742	1.000000	0.983607	0.998782
9	pill_IO	29	34	0.735294	0.862069	0.793651	0.984166
10	leather_IO	28	27	1.000000	0.964286	0.981818	0.998782
11	grid_IO	28	27	0.851852	0.821429	0.836364	0.989038
12	pill_NIO	28	23	0.826087	0.678571	0.745098	0.984166
13	cable_IO	28	32	0.843750	0.964286	0.900000	0.992692
14	wood_NIO	27	27	1.000000	1.000000	1.000000	1.000000
15	zipper_IO	27	29	0.931034	1.000000	0.964286	0.997564
16	transistor_IO	27	34	0.794118	1.000000	0.885246	0.991474
17	wood_IO	27	27	1.000000	1.000000	1.000000	1.000000
18	transistor_NIO	27	20	1.000000	0.740741	0.851064	0.991474
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	26	0.807692	0.840000	0.823529	0.989038
21	cable_NIO	25	21	0.952381	0.800000	0.869565	0.992692
22	metal_nut_IO	24	26	0.923077	1.000000	0.960000	0.997564
23	capsule_IO	24	23	0.913043	0.875000	0.893617	0.993910
24	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998782
25	capsule_NIO	22	23	0.869565	0.909091	0.888889	0.993910
26	bottle_NIO	22	21	1.000000	0.954545	0.976744	0.998782
27	metal_nut_NIO	22	20	1.000000	0.909091	0.952381	0.997564
28	toothbrush_IO	7	6	1.000000	0.857143	0.923077	0.998782
29	toothbrush_NIO	6	7	0.857143	1.000000	0.923077	0.998782
30	ALL	821	821	0.916869	0.910064	0.908737	0.908648

▼ MobileNetV3Small-Backbone x Dense-Head


```

epochs = 30
seed = 43
lr = 1e-3

model = make_premodel("mobilenetv3small", "dense")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy",
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)

```

► Training-Log

```

model = import_model(
    "https://drive.google.com/file/d/1ZEWbd1BzHmcBdFBaaBnY5xLtU0xCbmSa/view?usp=sharing",
    "mobilenetv3small_dense_bundle")

df = eval(model, "data/data_augmented_split/val")
df

```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	46	1.000000	0.920000	0.958333	0.995128
1	hazelnut_IO	43	47	0.914894	1.000000	0.955556	0.995128
2	screw_IO	36	39	0.846154	0.916667	0.880000	0.989038
3	carpet_NIO	35	24	1.000000	0.685714	0.813559	0.986602
4	tile_NIO	32	29	1.000000	0.906250	0.950820	0.996346
5	carpet_IO	31	42	0.738095	1.000000	0.849315	0.986602
6	zipper_NIO	31	27	1.000000	0.870968	0.931034	0.995128
7	screw_NIO	31	28	0.892857	0.806452	0.847458	0.989038
8	leather_NIO	30	31	0.967742	1.000000	0.983607	0.998782
9	pill_IO	29	32	0.718750	0.793103	0.754098	0.981730
10	leather_IO	28	27	1.000000	0.964286	0.981818	0.998782
11	grid_IO	28	19	0.947368	0.642857	0.765957	0.986602
12	pill_NIO	28	25	0.760000	0.678571	0.716981	0.981730
13	cable_IO	28	30	0.866667	0.928571	0.896552	0.992692
14	wood_NIO	27	26	1.000000	0.962963	0.981132	0.998782
15	zipper_IO	27	31	0.870968	1.000000	0.931034	0.995128
16	transistor_IO	27	40	0.675000	1.000000	0.805970	0.984166
17	wood_IO	27	28	0.964286	1.000000	0.981818	0.998782
18	transistor_NIO	27	14	1.000000	0.518519	0.682927	0.984166
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	34	0.705882	0.960000	0.813559	0.986602
21	cable_NIO	25	23	0.913043	0.840000	0.875000	0.992692
22	metal_nut_IO	24	31	0.774194	1.000000	0.872727	0.991474
23	capsule_IO	24	35	0.657143	0.958333	0.779661	0.984166
24	bottle_IO	23	24	0.958333	1.000000	0.978723	0.998782
25	capsule_NIO	22	11	0.909091	0.454545	0.606061	0.984166
26	bottle_NIO	22	21	1.000000	0.954545	0.976744	0.998782
27	metal_nut_NIO	22	15	1.000000	0.681818	0.810811	0.991474
28	toothbrush_IO	7	6	1.000000	0.857143	0.923077	0.998782
29	toothbrush_NIO	6	7	0.857143	1.000000	0.923077	0.998782
30	ALL	821	821	0.894472	0.876710	0.872429	0.878197

▽ MobileNetV3Small-Backbone x 2-Heads-Head

```
epochs = 30
seed = 43
```

```
lr = 1e-3

model = make_premodel("mobilenetv3small", "two_tasks")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss={"category": "sparse_categorical_crossentropy", "io": "sparse_categorical_crossentropy"},
    metrics={"category": ["accuracy"], "io": ["accuracy"]},
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
)

out = train_2heads(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/18cM72vmB1RqJmib2DNfh3OanucqnB4my/view?usp=sharing",
    "mobilenetv3small_two_tasks_bundle")

wrapped = TwoHeadTo30(model, categories)
df = eval(wrapped, "data/data_augmented_split/val", class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	51	0.980392	1.000000	0.990099	0.998782
1	hazelnut_IO	43	42	1.000000	0.976744	0.988235	0.998782
2	screw_IO	36	35	0.857143	0.833333	0.845070	0.986602
3	carpet_NIO	35	36	0.833333	0.857143	0.845070	0.986602
4	tile_NIO	32	30	0.966667	0.906250	0.935484	0.995128
5	carpet_IO	31	30	0.833333	0.806452	0.819672	0.986602
6	zipper_NIO	31	27	1.000000	0.870968	0.931034	0.995128
7	screw_NIO	31	32	0.812500	0.838710	0.825397	0.986602
8	leather_NIO	30	31	0.967742	1.000000	0.983607	0.998782
9	pill_IO	29	23	0.782609	0.620690	0.692308	0.980512
10	cable_IO	28	34	0.823529	1.000000	0.903226	0.992692
11	leather_IO	28	27	1.000000	0.964286	0.981818	0.998782
12	grid_IO	28	16	1.000000	0.571429	0.727273	0.985384
13	pill_NIO	28	34	0.676471	0.821429	0.741935	0.980512
14	wood_IO	27	25	1.000000	0.925926	0.961538	0.997564
15	zipper_IO	27	31	0.870968	1.000000	0.931034	0.995128
16	wood_NIO	27	29	0.931034	1.000000	0.964286	0.997564
17	transistor_NIO	27	21	1.000000	0.777778	0.875000	0.992692
18	transistor_IO	27	33	0.818182	1.000000	0.900000	0.992692
19	tile_IO	26	28	0.892857	0.961538	0.925926	0.995128
20	grid_NIO	25	37	0.675676	1.000000	0.806452	0.985384
21	cable_NIO	25	19	1.000000	0.760000	0.863636	0.992692
22	metal_nut_IO	24	26	0.884615	0.958333	0.920000	0.995128
23	capsule_IO	24	1	1.000000	0.041667	0.080000	0.971985
24	bottle_IO	23	26	0.884615	1.000000	0.938776	0.996346
25	capsule_NIO	22	45	0.488889	1.000000	0.656716	0.971985
26	bottle_NIO	22	19	1.000000	0.863636	0.926829	0.996346
27	metal_nut_NIO	22	20	0.950000	0.863636	0.904762	0.995128
28	toothbrush_IO	7	6	1.000000	0.857143	0.923077	0.998782
29	toothbrush_NIO	6	7	0.857143	1.000000	0.923077	0.998782
30	ALL	821	821	0.892923	0.869236	0.857045	0.872107

✕ Xception-Backbone x Direct-Head

```
epochs = 30
seed = 43
lr = 1e-3
```

```
model = make_premodel("xception", "direct")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy",
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/1FRBGnLc5Sj268LJ1_9dHD_KWVLI6Ks_s/view?usp=sharing",
    "xception_direct_bundle")

df = eval(model, "data/data_augmented_split/val")
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	38	0.947368	0.720000	0.818182	0.980512
1	hazelnut_IO	43	55	0.745455	0.953488	0.836735	0.980512
2	screw_IO	36	35	0.771429	0.750000	0.760563	0.979294
3	carpet_NIO	35	31	0.935484	0.828571	0.878788	0.990256
4	tile_NIO	32	33	0.969697	1.000000	0.984615	0.998782
5	carpet_IO	31	35	0.828571	0.935484	0.878788	0.990256
6	zipper_NIO	31	32	0.937500	0.967742	0.952381	0.996346
7	screw_NIO	31	32	0.718750	0.741935	0.730159	0.979294
8	leather_NIO	30	30	1.000000	1.000000	1.000000	1.000000
9	pill_IO	29	34	0.764706	0.896552	0.825397	0.986602
10	leather_IO	28	28	1.000000	1.000000	1.000000	1.000000
11	grid_IO	28	28	0.750000	0.750000	0.750000	0.982948
12	pill_NIO	28	23	0.869565	0.714286	0.784314	0.986602
13	cable_IO	28	33	0.818182	0.964286	0.885246	0.991474
14	wood_NIO	27	26	1.000000	0.962963	0.981132	0.998782
15	zipper_IO	27	26	0.961538	0.925926	0.943396	0.996346
16	transistor_IO	27	27	1.000000	1.000000	1.000000	1.000000
17	wood_IO	27	28	0.964286	1.000000	0.981818	0.998782
18	transistor_NIO	27	27	1.000000	1.000000	1.000000	1.000000
19	tile_IO	26	25	1.000000	0.961538	0.980392	0.998782
20	grid_NIO	25	25	0.720000	0.720000	0.720000	0.982948
21	cable_NIO	25	20	0.950000	0.760000	0.844444	0.991474
22	metal_nut_IO	24	26	0.884615	0.958333	0.920000	0.995128
23	capsule_IO	24	27	0.703704	0.791667	0.745098	0.984166
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	19	0.736842	0.636364	0.682927	0.984166
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	20	0.950000	0.863636	0.904762	0.995128
28	toothbrush_IO	7	7	1.000000	1.000000	1.000000	1.000000
29	toothbrush_NIO	6	6	1.000000	1.000000	1.000000	1.000000
30	ALL	821	821	0.897590	0.893426	0.892971	0.884287

✓ Xception-Backbone x Dense-Head

```
epochs = 30
seed = 43
```



```
lr = 1e-3

model = make_premodel("xception", "dense")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy",
    class_names=CLASS_NAMES_30,
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/1L9aMbt_t5wALwHNjdWwBu1IV7AFG0ER2/view?usp=sharing",
    "xception_dense_bundle")

df = eval(model, "data/data_augmented_split/val", class_names=CLASS_NAMES_30,)
df
```


	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	35	0.971429	0.680000	0.800000	0.979294
1	hazelnut_IO	43	58	0.724138	0.976744	0.831683	0.979294
2	screw_IO	36	39	0.794872	0.861111	0.826667	0.984166
3	carpet_NIO	35	35	0.942857	0.942857	0.942857	0.995128
4	tile_NIO	32	32	1.000000	1.000000	1.000000	1.000000
5	carpet_IO	31	31	0.935484	0.935484	0.935484	0.995128
6	zipper_NIO	31	33	0.909091	0.967742	0.937500	0.995128
7	screw_NIO	31	28	0.821429	0.741935	0.779661	0.984166
8	leather_NIO	30	30	1.000000	1.000000	1.000000	1.000000
9	pill_IO	29	34	0.764706	0.896552	0.825397	0.986602
10	cable_IO	28	35	0.771429	0.964286	0.857143	0.989038
11	leather_IO	28	28	1.000000	1.000000	1.000000	1.000000
12	grid_IO	28	31	0.774194	0.857143	0.813559	0.986602
13	pill_NIO	28	23	0.869565	0.714286	0.784314	0.986602
14	wood_IO	27	28	0.964286	1.000000	0.981818	0.998782
15	zipper_IO	27	25	0.960000	0.888889	0.923077	0.995128
16	wood_NIO	27	26	1.000000	0.962963	0.981132	0.998782
17	transistor_NIO	27	26	1.000000	0.962963	0.981132	0.998782
18	transistor_IO	27	28	0.964286	1.000000	0.981818	0.998782
19	tile_IO	26	26	1.000000	1.000000	1.000000	1.000000
20	grid_NIO	25	22	0.818182	0.720000	0.765957	0.986602
21	cable_NIO	25	18	0.944444	0.680000	0.790698	0.989038
22	metal_nut_IO	24	27	0.851852	0.958333	0.901961	0.993910
23	capsule_IO	24	25	0.720000	0.750000	0.734694	0.984166
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	21	0.714286	0.681818	0.697674	0.984166
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	19	0.947368	0.818182	0.878049	0.993910
28	toothbrush_IO	7	5	1.000000	0.714286	0.833333	0.997564
29	toothbrush_NIO	6	8	0.750000	1.000000	0.857143	0.997564
30	ALL	821	821	0.897130	0.889186	0.888092	0.889160

Xception-Backbone x 2-Heads-Head

```
epochs = 30
seed = 43
```



```
lr = 1e-3

model = make_premodel("xception", "two_tasks")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss={"category": "sparse_categorical_crossentropy", "io": "sparse_categorical_crossentropy"},
    metrics={"category": ["accuracy"], "io": ["accuracy"]},
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
)

out = train_2heads(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/1dHpyI8Y2udgtP_v-kyW75_vbBEJWhXDf/view?usp=sharing",
    "xception_two_tasks_bundle")

wrapped = TwoHeadTo30(model, categories)
df = eval(wrapped, "data/data_augmented_split/val", class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	37	0.972973	0.720000	0.827586	0.981730
1	hazelnut_IO	43	56	0.750000	0.976744	0.848485	0.981730
2	screw_IO	36	35	0.771429	0.750000	0.760563	0.979294
3	carpet_NIO	35	34	0.970588	0.942857	0.956522	0.996346
4	tile_NIO	32	33	0.969697	1.000000	0.984615	0.998782
5	carpet_IO	31	32	0.937500	0.967742	0.952381	0.996346
6	zipper_NIO	31	33	0.909091	0.967742	0.937500	0.995128
7	screw_NIO	31	32	0.718750	0.741935	0.730159	0.979294
8	leather_NIO	30	30	1.000000	1.000000	1.000000	1.000000
9	pill_IO	29	30	0.733333	0.758621	0.745763	0.981730
10	cable_IO	28	35	0.771429	0.964286	0.857143	0.989038
11	leather_IO	28	28	1.000000	1.000000	1.000000	1.000000
12	grid_IO	28	28	0.750000	0.750000	0.750000	0.982948
13	pill_NIO	28	27	0.740741	0.714286	0.727273	0.981730
14	wood_IO	27	28	0.964286	1.000000	0.981818	0.998782
15	zipper_IO	27	25	0.960000	0.888889	0.923077	0.995128
16	wood_NIO	27	26	1.000000	0.962963	0.981132	0.998782
17	transistor_NIO	27	27	1.000000	1.000000	1.000000	1.000000
18	transistor_IO	27	27	1.000000	1.000000	1.000000	1.000000
19	tile_IO	26	25	1.000000	0.961538	0.980392	0.998782
20	grid_NIO	25	25	0.720000	0.720000	0.720000	0.982948
21	cable_NIO	25	18	0.944444	0.680000	0.790698	0.989038
22	metal_nut_IO	24	26	0.884615	0.958333	0.920000	0.995128
23	capsule_IO	24	25	0.680000	0.708333	0.693878	0.981730
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	21	0.666667	0.636364	0.651163	0.981730
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	20	0.950000	0.863636	0.904762	0.995128
28	toothbrush_IO	7	5	1.000000	0.714286	0.833333	0.997564
29	toothbrush_NIO	6	8	0.750000	1.000000	0.857143	0.997564
30	ALL	821	821	0.883851	0.878285	0.877179	0.878197

✓ VGG16-Backbone x Direct-Head

```
epochs = 30
seed = 43
```

```
lr = 1e-3

model = make_premodel("vgg16", "direct")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy",
    class_names=CLASS_NAMES_30,
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/18Dmcc6enkIM5qPug6rGEx5hf_Xb3PPLL/view?usp=sharing",
    "vgg16_direct_bundle")

df = eval(model, "data/data_augmented_split/val", class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	45	0.955556	0.860000	0.905263	0.989038
1	hazelnut_IO	43	48	0.854167	0.953488	0.901099	0.989038
2	screw_IO	36	38	0.894737	0.944444	0.918919	0.992692
3	carpet_NIO	35	27	1.000000	0.771429	0.870968	0.990256
4	tile_NIO	32	29	1.000000	0.906250	0.950820	0.996346
5	carpet_IO	31	39	0.794872	1.000000	0.885714	0.990256
6	zipper_NIO	31	30	0.933333	0.903226	0.918033	0.993910
7	screw_NIO	31	29	0.931034	0.870968	0.900000	0.992692
8	leather_NIO	30	27	1.000000	0.900000	0.947368	0.996346
9	pill_IO	29	28	0.714286	0.689655	0.701754	0.979294
10	cable_IO	28	30	0.866667	0.928571	0.896552	0.992692
11	leather_IO	28	31	0.903226	1.000000	0.949153	0.996346
12	grid_IO	28	31	0.806452	0.892857	0.847458	0.989038
13	pill_NIO	28	29	0.689655	0.714286	0.701754	0.979294
14	wood_IO	27	27	1.000000	1.000000	1.000000	1.000000
15	zipper_IO	27	28	0.892857	0.925926	0.909091	0.993910
16	wood_NIO	27	27	1.000000	1.000000	1.000000	1.000000
17	transistor_NIO	27	27	0.962963	0.962963	0.962963	0.997564
18	transistor_IO	27	27	0.962963	0.962963	0.962963	0.997564
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	22	0.863636	0.760000	0.808511	0.989038
21	cable_NIO	25	23	0.913043	0.840000	0.875000	0.992692
22	metal_nut_IO	24	25	0.880000	0.916667	0.897959	0.993910
23	capsule_IO	24	26	0.769231	0.833333	0.800000	0.987820
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	20	0.800000	0.727273	0.761905	0.987820
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	21	0.904762	0.863636	0.883721	0.993910
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.998782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.902166	0.898709	0.898162	0.897686

VGG16-Backbone x Dense-Head

```
epochs = 30
seed = 43
```

```
lr = 1e-3

model = make_premodel("vgg16", "dense")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
    callback_metric="val_accuracy",
    class_names=CLASS_NAMES_30,
)

out = train(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
zip_path = export_model(run)
print("Exported:", zip_path)
```

► Training-Log

```
model = import_model(
    "https://drive.google.com/file/d/1PElho0Uwpo7AArGtNugX7c3IeV5MJRN0/view?usp=sharing",
    "vgg16_dense_bundle")

df = eval(model, "data/data_augmented_split/val", class_names=CLASS_NAMES_30)
df
```

	class	true_count	pred_count	precision	recall	f1	accuracy
0	hazelnut_NIO	50	45	0.955556	0.860000	0.905263	0.989038
1	hazelnut_IO	43	48	0.854167	0.953488	0.901099	0.989038
2	screw_IO	36	37	0.864865	0.888889	0.876712	0.989038
3	carpet_NIO	35	33	0.939394	0.885714	0.911765	0.992692
4	tile_NIO	32	29	1.000000	0.906250	0.950820	0.996346
5	carpet_IO	31	33	0.878788	0.935484	0.906250	0.992692
6	zipper_NIO	31	32	0.875000	0.903226	0.888889	0.991474
7	screw_NIO	31	30	0.866667	0.838710	0.852459	0.989038
8	leather_NIO	30	27	1.000000	0.900000	0.947368	0.996346
9	pill_IO	29	30	0.700000	0.724138	0.711864	0.979294
10	cable_IO	28	31	0.870968	0.964286	0.915254	0.993910
11	leather_IO	28	31	0.903226	1.000000	0.949153	0.996346
12	grid_IO	28	26	0.846154	0.785714	0.814815	0.987820
13	pill_NIO	28	27	0.703704	0.678571	0.690909	0.979294
14	wood_IO	27	27	1.000000	1.000000	1.000000	1.000000
15	zipper_IO	27	26	0.884615	0.851852	0.867925	0.991474
16	wood_NIO	27	27	1.000000	1.000000	1.000000	1.000000
17	transistor_NIO	27	27	0.962963	0.962963	0.962963	0.997564
18	transistor_IO	27	27	0.962963	0.962963	0.962963	0.997564
19	tile_IO	26	29	0.896552	1.000000	0.945455	0.996346
20	grid_NIO	25	27	0.777778	0.840000	0.807692	0.987820
21	cable_NIO	25	22	0.954545	0.840000	0.893617	0.993910
22	metal_nut_IO	24	28	0.857143	1.000000	0.923077	0.995128
23	capsule_IO	24	30	0.733333	0.916667	0.814815	0.987820
24	bottle_IO	23	23	1.000000	1.000000	1.000000	1.000000
25	capsule_NIO	22	16	0.875000	0.636364	0.736842	0.987820
26	bottle_NIO	22	22	1.000000	1.000000	1.000000	1.000000
27	metal_nut_NIO	22	18	1.000000	0.818182	0.900000	0.995128
28	toothbrush_IO	7	8	0.875000	1.000000	0.933333	0.998782
29	toothbrush_NIO	6	5	1.000000	0.833333	0.909091	0.998782
30	ALL	821	821	0.901279	0.896226	0.896013	0.895250

✓ VGG16-Backbone x 2-Heads-Head

```
epochs = 30
seed = 43
```



```
lr = 1e-3

model = make_premodel("vgg16", "two_tasks")
model.compile(
    optimizer=keras.optimizers.Adam(lr),
    loss={"category": "sparse_categorical_crossentropy", "io": "sparse_categorical_crossentropy"},
    metrics={"category": ["accuracy"], "io": ["accuracy"]},
)

inp = TrainInput(
    data_dir="data/data_augmented_split",
    model=model,
    seed=seed,
    epochs=epochs,
    tensorboard_logdir=f"logs/{model.name}",
)

out = train_2heads(inp)

run = dict(name=out.model.name, model=out.model, model_name=out.model.name, lr=lr, seed=seed, epochs=epochs)
```